InsaCloud

Construire un mini-fournisseur de cloud, de zéro

Guide technique complet

De la théorie au déploiement, pour qui n'a jamais fait ça



Le contrôleur n'exécute aucune machine louée : il n'a que le client Docker. Chaque worker s'inscrit seul au démarrage, puis signale sa présence. Sans battement de cœur il passe hors ligne, et ses machines sont recréées sur un nœud disponible.

Ce document explique *tout* un projet réel : une plateforme qui loue des machines Linux à la minute, accessibles en SSH, en terminal web ou en bureau graphique dans le navigateur. On y part des notions — qu'est-ce qu'un conteneur, pourquoi un proxy inverse, ce que veut dire « idempotent » — pour arriver au code exact qui tourne en production.

Projet 3 — *Outils de déploiement de plateformes*
INSA Centre-Val de Loire, STI 4A FISE, 2026/2027 — Dr. Nadia Fettah

13 octobre 2026

Table des matières

Avant-propos 1

I Les notions 4

1 Le problème : louer des machines 5
1.1 Ce qu'on nous demande . . . 5
1.2 La décision qui structure tout : conteneurs plutôt que VM . . 5
1.3 Les questions auxquelles il faut répondre . . . 6

2 Conteneurs : ce qui se passe réellement 8
2.1 Trois mécanismes du noyau Linux . . . 8
2.2 Les images et le système de fichiers en couches . . . 9
2.3 Les commandes qu'il faut connaître . . . 9

3 Une application web, de bout en bout 11
3.1 Ce qui se passe quand on tape une adresse . . . 11
3.2 Flask en dix minutes . . . 12
3.3 Le même chemin pour un humain et pour un programme . . 13

4 L'infrastructure décrite dans des fichiers 14
4.1 Le problème du « ça marche chez moi » . . . 14
4.2 Vagrant : des VM reproductibles . . . 14
4.3 Ansible : décrire l'état, pas les étapes . . . 15

5 DevSecOps : pourquoi une chaîne automatique 17
5.1 Le constat . . . 17
5.2 Les familles de contrôles . . . 17

II L'architecture 19

6 La carte du territoire 20
6.1 Un contrôleur, trois exécutants . . . 20
6.2 Comment le contrôleur pilote les workers . . . 21
6.3 Comment le contrôleur apprend l'existence des workers . . . 21

7 Le modèle de données 23
7.1 Pourquoi séparer la machine de la location . . . 23
7.2 Les sept tables . . . 24
7.3 Deux index qui portent une règle métier . . . 24

8 Le cycle de vie d'une machine **26**
8.1 Les cinq étapes . . . 26
8.2 La branche qui fait la différence . . . 27
8.3 Trois niveaux de résilience . . . 27

III La construction **29**

9 Les images des machines louées **30**
9.1 Le problème : plusieurs services dans un conteneur . . . 30
9.2 Trois Dockerfiles, six images . . . 30
9.3 Le script de démarrage . . . 31
9.4 Rotation du mot de passe à chaud . . . 32

10 Piloter Docker depuis Python **33**
10.1 Pourquoi la ligne de commande et pas un SDK . . . 33
10.2 La création d'une machine . . . 34
10.3 Le durcissement, option par option . . . 34
10.4 Reconnaître ses machines : l'erreur qui a tout détruit . . . 35

11 La base de données **37**
11.1 SQLAlchemy : pourquoi un ORM . . . 37
11.2 Alembic : faire évoluer un schéma sans perdre les données . . . 38
11.3 La course au démarrage : un bug qui ne se voit qu'en production 38
11.4 Ne jamais journaliser un secret . . . 39

12 L'application et son API **40**
12.1 Une organisation en couches . . . 40
12.2 Le résultat d'une opération . . . 40
12.3 Les points d'entrée imposés . . . 40
12.4 Protéger les routes des agents . . . 41

13 Le parc de nœuds **43**
13.1 L'agent, en bibliothèque standard seulement . . . 43
13.2 Le battement de cœur, et les trois états . . . 43
13.3 Démarrer avant le contrôleur . . . 44
13.4 L'ordonnanceur . . . 45

14 Le Faucheur **46**
14.1 Un démon, trois missions . . . 46
14.2 La réconciliation, ou l'art de douter de soi . . . 46
14.3 L'erreur du `+-1 minutes` . . . 47

15 La sécurité **48**
15.1 Deux clés, jamais la même . . . 48
15.2 Le mot de passe root : montré une fois, conservé chiffré . . . 48
15.3 CSRF : empêcher un autre site d'agir en votre nom . . . 49
15.4 Empreinte et chiffrement, à ne pas confondre . . . 50
15.5 Les en-têtes et la politique de contenu . . . 51

16 La haute disponibilité **52**
16.1 Le scénario exact du cahier des charges . . . 52
16.2 Le compteur de migrations . . . 53

17 Sauvegarder le travail avant destruction **54**
17.1 Le besoin . . . 54
17.2 Le principe : un flux, pas un fichier temporaire . . . 54
17.3 La règle qui prime sur tout . . . 55
17.4 Retenter sans boucler . . . 55

IV Le déploiement **57**

18 Monter le parc avec Vagrant **58**
18.1 Quatre machines décrites en un fichier . . . 58
18.2 Plusieurs providers, et pourquoi . . . 58
18.3 Les commandes du quotidien . . . 59

19 Configurer avec Ansible **60**
19.1 La partition : quatre plays, neuf rôles . . . 60
19.2 Un piège réel : la déduplication des rôles . . . 61
19.3 Les secrets : Ansible Vault . . . 62
19.4 Prouver l'idempotence . . . 62
19.5 L'inventaire dynamique . . . 63

20 Servir l'application **64**
20.1 nginx devant Gunicorn . . . 64
20.2 systemd : faire tourner les services . . . 64
20.3 Le watchdog . . . 65

21 La pile `docker compose` **66**
21.1 Pourquoi une seconde façon de déployer . . . 66
21.2 Un bloc partagé par deux services . . . 66
21.3 Le volume partagé, et pourquoi il ne s'invente pas . . . 67
21.4 Les profils . . . 68

V La vérification **69**

22 Les tests **70**
22.1 Tester Docker sans Docker 70
22.2 Partir d'une base vierge à chaque test 70
22.3 Préparer l'environnement avant l'import 71
22.4 Ce que couvrent les 168 tests 72
22.5 Le test de cloisonnement, le plus important 72
22.6 Les tests d'intégration : la vraie pile 73

23 La chaîne d'intégration **74**
23.1 Onze travaux 74
23.2 Le verrou 75
23.3 Trois incidents instructifs 75
23.4 Empêcher la documentation de vieillir 76

24 Ce qu'on a trouvé en faisant tourner **77**
24.1 Ce qui a été vérifié sur un cluster réel 78

VI Annexes **79**

25 Refaire le projet depuis zéro **80**

26 Commandes utiles **81**
26.1 Le projet 81
26.2 Diagnostiquer 81

27 Erreurs fréquentes **83**

28 Conformité au cahier des charges **84**

29 Glossaire **86**

30 Pour aller plus loin **88**
30.1 Ce que ce projet ne fait pas 88
30.2 Les suites naturelles 88
30.3 Le dépôt 88

Avant-propos

Ce que vous allez comprendre

Un utilisateur arrive sur un site web, crée un compte, choisit « Ubuntu, bureau graphique, 30 minutes », clique. Quinze secondes plus tard il a une machine Linux à lui, avec une adresse SSH, un terminal dans son navigateur et un bureau XFCE complet. Trente minutes plus tard, la machine a disparu toute seule et le port qu'elle occupait est rendu.

Derrière ce clic, il y a un serveur web, une base de données, un démon Docker sur trois machines différentes, un ordonnanceur qui choisit laquelle, un service qui surveille les horloges, des certificats, un pare-feu, un coffre à secrets, et une chaîne d'intégration qui refuse de publier quoi que ce soit tant que onze contrôles ne sont pas au vert.

Ce document explique chacune de ces pièces. Pas en survol : le code exact, les raisons de chaque choix, et les erreurs commises en chemin.

> **À retenir**
>
> À la fin de ce guide, vous devez être capable de reconstruire un projet de ce type sans le recopier. C'est pour cela que chaque chapitre explique d'abord *pourquoi* avant de montrer *comment*.

À qui s'adresse ce guide

À quelqu'un qui sait programmer un peu, qui a déjà vu une ligne de commande, et qui n'a jamais construit d'infrastructure. Aucune connaissance préalable de Docker, d'Ansible, de Vagrant, de Flask ou de la sécurité web n'est supposée.

Chaque notion nouvelle est introduite dans un encadré bleu avant d'être utilisée. Les pièges réellement rencontrés sont signalés en orange. Les points à retenir sont en vert.

> **Notion — Comment lire les extraits de code**
>
> Les extraits sont tirés du dépôt réel, sans retouche. Quand un fichier est trop long pour tenir ici, seules les lignes qui portent l'idée sont reproduites et les coupures sont marquées. Le chemin du fichier est toujours donné : vous pouvez ouvrir l'original pour voir le reste.

Le résultat, en images

Avant de plonger dans la mécanique, voici ce que l'utilisateur voit.

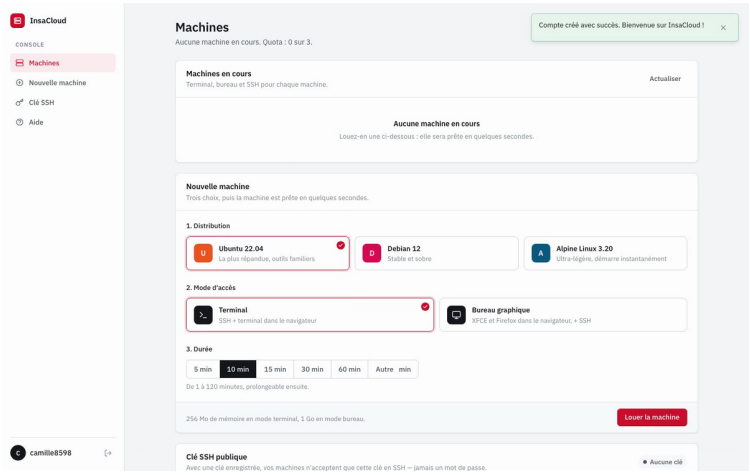


Figure 1 – Le tableau de bord. Les machines en cours, leur temps restant, et le formulaire de location. Tout tient sur une page : c'est un choix, pas une limite.

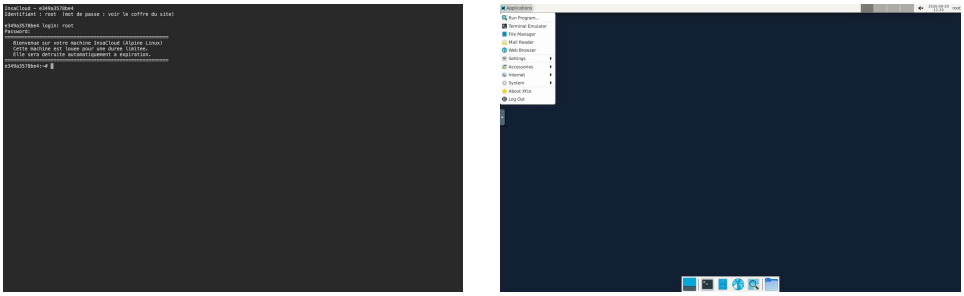


Figure 2 – À gauche, un terminal dans le navigateur, sans rien installer. À droite, un bureau XFCE complet avec Firefox, également dans le navigateur. La même machine peut offrir l'un ou l'autre selon le mode choisi à la location.

Comment ce guide est organisé

Partie I — Les notions Ce qu'il faut savoir avant d'ouvrir le code : virtualisation, conteneurs, modèle web, infrastructure décrite en fichiers, et ce que « DevSecOps » veut dire concrètement.

Partie II — L'architecture La carte du territoire : qui parle à qui, où vivent les données, comment une machine naît et meurt.

Partie III — La construction Le cœur du guide. Chaque composant, son code, et les décisions qui l'ont façonné.

Partie IV — Le déploiement Comment tout cela arrive sur de vraies machines, en une commande, de façon reproductible.

Partie V — La vérification Les tests, la chaîne d'intégration, et le récit des défauts trouvés en faisant tourner le projet plutôt qu'en le relisant.

Annexes Commandes utiles, glossaire, erreurs fréquentes, et la grille de conformité au cahier des charges.

Le dépôt

Tout le code commenté ici est public : `https://github.com/aziz3r/insacloud`. Les chemins de fichiers mentionnés dans le guide sont relatifs à la racine du dépôt.

Composant	**Lignes**
Application Flask (`2_webapp/`)	environ 3 500
Tests (`2_webapp/tests/`)	environ 1 600
Automatisation Ansible (`3_ansible/`)	environ 1 900
Chaîne d'intégration (`.github/workflows/ci.yml`)	601
Images des machines (`1_docker/`)	environ 400
Infrastructure (`Vagrantfile`, `docker-compose.yml`)	333
Total	**environ 10 700**

Première partie

Les notions

CHAPITRE 1

Le problème : louer des machines

1.1 Ce qu'on nous demande

L'énoncé tient en une phrase : construire une plateforme où un utilisateur s'inscrit, se connecte, loue une ou plusieurs instances Linux de différentes distributions pour une durée qu'il choisit, y accède en SSH, et où le service reste disponible même si une machine du parc tombe.

Derrière cette phrase se cachent dix-sept blocs de travail que le cahier des charges détaille : l'environnement de développement, l'API web, la base de données, la conteneurisation, l'infrastructure virtuelle, l'automatisation, un agent sur chaque nœud, un ordonnanceur, le service de location, le déploiement des conteneurs, l'accès SSH, l'expiration, la haute disponibilité, la sécurité, la chaîne d'intégration, les tests, et la documentation.

> **Notion — Le vocabulaire du cloud**
>
> Un **fournisseur de cloud** vend de la capacité de calcul à la demande. On distingue trois niveaux de service :
>
> **IaaS** *Infrastructure as a Service* — on vous loue une machine nue, vous y installez ce que vous voulez. C'est Amazon EC2, et c'est ce que nous construisons.
>
> **PaaS** *Platform as a Service* — on vous loue un endroit où déposer votre code, sans vous occuper de la machine. C'est Heroku.
>
> **SaaS** *Software as a Service* — on vous loue un logiciel fini. C'est Gmail.
>
> Notre plateforme est donc un mini-IaaS : l'utilisateur reçoit une machine et un accès root dessus.

1.2 La décision qui structure tout : conteneurs plutôt que VM

Louer « une machine Linux » peut se faire de deux façons.

Une machine virtuelle. On émule un ordinateur entier : un noyau Linux complet, son propre espace mémoire, ses disques virtuels. C'est ce que fait VirtualBox ou VMware. L'isolation est quasi totale, mais démarrer une VM

prend trente secondes à une minute, et chacune réserve plusieurs centaines de mégaoctets rien que pour son noyau.

Un conteneur. On ne démarre pas de second noyau : on réutilise celui de la machine hôte, en cloisonnant ce que le processus peut voir. Le démarrage prend moins d'une seconde et le surcoût mémoire est celui des processus eux-mêmes.

Pour une plateforme qui loue à la minute, la différence est décisive : un utilisateur n'attend pas une minute après avoir cliqué. Le cahier des charges tranche d'ailleurs explicitement — « des conteneurs dans notre cas ».

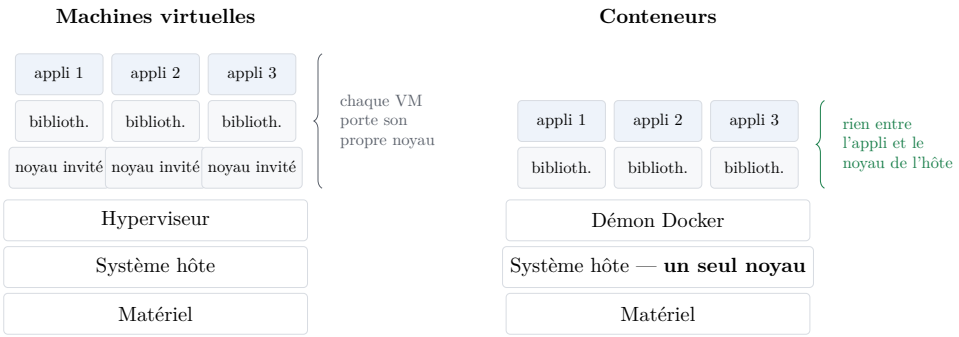


Figure 1.1 – La seule vraie différence : le conteneur n'embarque pas de noyau. Tout le reste — rapidité de démarrage, consommation mémoire, densité — en découle.

Piège — « Un conteneur, c'est une VM légère »

C'est l'image qu'on donne souvent, et elle induit en erreur. Un conteneur *partage le noyau de l'hôte*. Conséquences concrètes :

- on ne peut pas faire tourner un conteneur Windows sur un noyau Linux ;
- une faille d'évasion du noyau compromet l'hôte *et* tous les autres conteneurs — c'est pourquoi nous retirons les capacités Linux inutiles (chapitre 15) ;
- un conteneur n'a pas de `init` par défaut : pour y faire tourner plusieurs services, il faut en installer un (chapitre 9).

1.3 Les questions auxquelles il faut répondre

Dire « on crée un conteneur par location » ne suffit pas. Il faut résoudre ce que toute plateforme d'hébergement doit résoudre :

Où placer la machine ? S'il y a trois serveurs, lequel choisir ? Que faire si l'un ne répond plus ?

Comment y accéder ? Un conteneur n'a pas d'adresse publique. Il faut publier un port, et deux machines ne peuvent pas prendre le même.

Comment la faire disparaître ? Personne ne viendra cliquer sur « supprimer » à 3 h du matin. Il faut un service qui surveille les horloges.

Comment garder le secret ? Le mot de passe root doit être montré à l'utilisateur, mais ne doit pas traîner en clair dans une base.

Comment reconstruire tout cela ? Si le serveur brûle, combien de temps faut-il pour remonter la plateforme à l'identique ?

Chacune occupe un chapitre de la partie III.

CHAPITRE 2

Conteneurs : ce qui se passe réellement

2.1 Trois mécanismes du noyau Linux

Un conteneur n'est pas un objet du noyau. C'est un processus ordinaire auquel on a appliqué trois mécanismes.

Notion — Les espaces de noms (*namespaces*)

Un espace de noms cloisonne *ce qu'un processus peut voir*. Linux en propose plusieurs, dont :

PID le processus se croit seul ; il voit son propre arbre de processus, numéroté à partir de 1.

réseau il a ses propres interfaces, sa table de routage, ses ports. Le port 22 du conteneur n'est pas le port 22 de l'hôte.

montage il a sa propre vision du système de fichiers.

UTS il a son propre nom de machine.

C'est l'illusion d'être seul au monde.

Notion — Les groupes de contrôle (*cgroups*)

Les espaces de noms cloisonnent la *vue* ; les cgroups limitent la *consommation*. On y plafonne la mémoire, le temps processeur, le nombre de processus, les entrées-sorties. Sans cgroups, un conteneur parti en boucle infinie met à genoux tous les autres.

Dans notre projet, ces limites sont posées explicitement à la création : `--memory`, `--memory-reservation`, `--cpus`, `--pids-limit`.

Notion — Les capacités (*capabilities*)

Historiquement, un processus était soit root (tout permis) soit non-root (rien). Linux a découpé les pouvoirs de root en une quarantaine de *capacités* indépendantes : `CAP_NET_BIND_SERVICE` (écouter sous le port 1024), `CAP_SYS_ADMIN` (monter des systèmes de fichiers), etc.

On peut donc autoriser un processus à écouter sur le port 22 sans l'autoriser à charger un module noyau. C'est exactement ce que fait notre durcissement : retirer tout, puis rendre les dix capacités dont `sshd` a réellement besoin.

2.2 Les images et le système de fichiers en couches

Une **image** est un modèle figé, en lecture seule. Un **conteneur** est une instance en exécution de cette image, avec une fine couche inscriptible par-dessus.



Figure 2.1 – Une image est un empilement de couches. Dix conteneurs issus de la même image ne stockent la base qu'une fois : seule la couche du dessus leur est propre.

Cette architecture a une conséquence qu'on oublie vite : **détruire un conteneur détruit sa couche inscriptible**, donc tout ce que l'utilisateur y a écrit. C'est pourquoi le projet sauvegarde le travail avant destruction (chapitre 17) et pourquoi la reprise après panne ne rend pas les fichiers (chapitre 16).

> **À retenir**
>
> Une image, c'est la recette figée. Un conteneur, c'est le plat : on le mange, on jette l'assiette. Tout ce qui doit survivre est rangé ailleurs — dans un volume, dans une base, ou dans une archive.

2.3 Les commandes qu'il faut connaître

```
# Construire une image a partir du Dockerfile du dossier courant
docker build -t mon_image:latest .

# Lancer un conteneur en arriere-plan (-d), en publiant un port
docker run -d --name ma_machine -p 2222:22 mon_image:latest

# Lister ce qui tourne ; -a ajoute ce qui est arrete
docker ps
docker ps -a --filter "label=insacloud.role=rented-machine"

# Entrer dans un conteneur qui tourne
docker exec -it ma_machine /bin/bash
```

```
# Comprendre pourquoi il ne demarre pas
docker logs ma_machine

# Tout savoir de sa configuration reelle (ports, limites, capacites)
docker inspect ma_machine

# Le detruire, meme s'il tourne
docker rm -f ma_machine
```

Notion — Publier un port

`-p 2222:22` signifie : « ce qui arrive sur le port **2222 de l'hôte** est transmis au port **22 du conteneur** ». Le conteneur croit écouter sur 22 ; le monde extérieur frappe sur 2222.
C'est le mécanisme qui permet de faire tourner cent serveurs SSH sur une seule machine : chacun écoute son 22 interne, mais reçoit un port externe différent. Notre plateforme en attribue un dans la plage 8000–9000 et garantit l'unicité par un index en base (chapitre 11).

CHAPITRE 3

Une application web, de bout en bout

3.1 Ce qui se passe quand on tape une adresse



Figure 3.1 – La chaîne complète d'une requête. Chaque maillon a une raison d'être précise.

Pourquoi nginx devant ? Flask sait répondre à du HTTP, mais n'est pas prévu pour affronter Internet. On place donc un *proxy inverse* devant : nginx termine le TLS (le « s » de HTTPS), sert les fichiers statiques — images, CSS — sans réveiller Python, impose des en-têtes de sécurité, et protège l'application des requêtes malformées.

> **Notion — Proxy direct et proxy inverse**
>
> Un **proxy direct** agit pour le client : votre navigateur passe par lui pour atteindre le web. Un **proxy inverse** agit pour le serveur : le client croit parler au site, alors qu'il parle à nginx, qui relaie. Le premier cache qui demande, le second cache qui répond.

Pourquoi Gunicorn ? Le serveur de développement de Flask traite une requête à la fois et le dit lui-même : « ne pas utiliser en production ». Gunicorn lance plusieurs processus ouvriers qui se partagent les requêtes, redémarre ceux qui plantent, et sait s'arrêter proprement.

> **Notion — WSGI**
>
> *Web Server Gateway Interface* est le contrat entre un serveur Python et une application Python. L'application expose un objet appelable qui reçoit un dictionnaire décrivant la requête et renvoie une réponse. Grâce à ce contrat, on remplace Gunicorn par uWSGI ou Waitress sans toucher une ligne de Flask.

Piège — Un seul processus Gunicorn n'est pas une option

Avec un seul ouvrier, une requête lente — créer un conteneur prend plusieurs secondes — bloque *tous* les autres utilisateurs. Notre déploiement en lance quatre. Mais plusieurs processus qui démarrent simultanément posent un autre problème, celui de la course à l'initialisation de la base : nous y revenons au chapitre 11, car il nous a réellement coûté une panne.

3.2 Flask en dix minutes

Flask est un *micro*-framework : il fournit le routage, les gabarits et les sessions, et laisse le reste à des bibliothèques que vous choisissez. C'est l'inverse de Django, qui décide à votre place.

```
from flask import Flask, render_template, request, redirect, url_for,
    ↪ session

app = Flask(__name__)
app.secret_key = "..."           # sert a SIGNER les cookies de session

@app.route("/")                  # une URL ...
def accueil():                   # ... associee a une fonction
    return render_template("index.html", titre="Bonjour")

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        session["user"] = request.form["username"]   # cookie signe
        return redirect(url_for("accueil"))
    return render_template("login.html")
```

Le routage `@app.route` associe un chemin d'URL à une fonction. La fonction renvoie ce que le navigateur recevra.

Les gabarits `render_template` remplit un fichier HTML avec des variables, grâce au moteur Jinja2.

La session `session` est un dictionnaire stocké *dans un cookie* chez le client.

Piège — Une session Flask est signée, pas chiffrée

C'est l'erreur la plus courante, et nous l'avons commise. Le contenu d'un cookie de session est encodé en base64 : **n'importe qui peut le lire**, sans la clé. La signature empêche seulement de le *modifier*.
Nous y avions mis le mot de passe root d'une machine, le temps de l'afficher une fois. Il suffisait d'ouvrir les outils du navigateur pour le récupérer. La correction consiste à ne stocker que l'identifiant de la

machine et à relire le secret en base, chiffré (chapitre 15).

3.3 Le même chemin pour un humain et pour un programme

Un trait d'architecture qui revient partout dans ce projet : chaque route sert soit du HTML, soit du JSON, selon ce que l'appelant demande dans l'en-tête `Accept`. Il n'y a donc pas un site d'un côté et une API de l'autre, mais une seule logique métier avec deux rendus.

Listing 3.1 — `2_webapp/app.py` — choisir le format de la réponse

```
def veut_json() -> bool:
    # Un client qui accepte TOUT (Accept: */*, cas de curl sans en-tete
        ↪ )
    # n'est pas un client d'API : seul celui qui demande du JSON sans
        ↪ accepter
    # le HTML est traite comme tel.
    accepte = request.accept_mimetypes
    return accepte.accept_json and not accepte.accept_html
```

Piège — `Accept: */*` n'est pas une demande de JSON

Notre première version répondait du JSON à tout client acceptant `application/json` — ce que fait `*/*`. Résultat : certains navigateurs et tous les `curl` nus recevaient du JSON à la place de la page. La condition correcte est : *accepte le JSON* **et** *n'accepte pas le HTML.*

CHAPITRE 4

L'infrastructure décrite dans des fichiers

4.1 Le problème du « ça marche chez moi »

Un projet installé à la main sur un serveur a une propriété désastreuse : personne, pas même son auteur, ne sait exactement ce qui a été fait. Au remontage, il manque toujours un paquet, un fichier de configuration, une permission.

L'*infrastructure as code* consiste à écrire l'état voulu dans des fichiers versionnés, et à laisser un outil l'appliquer. Deux outils se partagent le travail.

Vagrant crée les machines virtuelles : combien, à partir de quelle image, avec quelle mémoire, quelle adresse.

Ansible les configure : installe les paquets, dépose les fichiers, démarre les services.

4.2 Vagrant : des VM reproductibles

Listing 4.1 – Le principe, en quelques lignes

```
Vagrant.configure("2") do |config|
  config.vm.box = "bento/ubuntu-22.04"       # l'image de base
  config.vm.define "worker1" do |node|
    node.vm.hostname = "worker1"
    node.vm.network "private_network", ip: "192.168.56.11"
    node.vm.provider "virtualbox" do |v|
      v.memory = 3072
      v.cpus   = 2
    end
  end
end
```

`vagrant up` lit ce fichier et crée la VM ; `vagrant destroy` l'efface. Entre les deux, l'état est entièrement décrit par le fichier : n'importe qui, sur n'importe quelle machine, obtient le même résultat.

> **Notion — *Box* et *provider***
>
> Une **box** est une image de VM préfabriquée, téléchargée depuis un ca-

talogue public. Un **provider** est le logiciel de virtualisation qui l'exécute : VirtualBox, VMware, Parallels, QEMU. Le même `Vagrantfile` peut cibler plusieurs providers — indispensable ici, car VirtualBox ne fonctionne pas sur les Mac à puce Apple.

4.3 Ansible : décrire l'état, pas les étapes

Ansible se connecte en SSH et applique des *tâches*. La différence avec un script shell est fondamentale.

Listing 4.2 – Un script shell décrit des actions

```
apt-get install -y nginx          # se comporte mal si deja installe
mkdir /etc/insacloud              # echoue si le dossier existe
echo "..." > /etc/insacloud/conf  # ecrase a chaque fois
systemctl restart nginx           # redemarre meme si rien n'a change
```

Listing 4.3 – Ansible décrit un état voulu

```
- name: nginx est installe
  ansible.builtin.apt:
    name: nginx
    state: present

- name: Le dossier de configuration existe, avec les bons droits
  ansible.builtin.file:
    path: /etc/insacloud
    state: directory
    owner: insacloud
    mode: "0750"

- name: La configuration est en place
  ansible.builtin.template:
    src: insacloud.conf.j2
    dest: /etc/insacloud/conf
  notify: Redemarrer nginx        # UNIQUEMENT si le fichier a change
```

Notion — Idempotence

Une opération est **idempotente** si l'appliquer deux fois donne le même résultat que l'appliquer une fois. C'est la propriété centrale d'Ansible : relancer le playbook sur un serveur déjà configuré ne doit *rien* changer. La preuve se lit dans le compte rendu : au second passage, `changed=0`. Dans ce projet, c'est mesuré sur les quatre nœuds à chaque modification (chapitre 19).

> **À retenir**
>
> Idempotent ne veut pas dire « ne fait rien la deuxième fois », mais « vérifie l'état, et n'agit que s'il diffère de celui qui est voulu ». La nuance compte : c'est elle qui permet de relancer le déploiement sans crainte, à n'importe quel moment.

Le vocabulaire d'Ansible.

Inventaire la liste des machines, rangées par groupes (`[controllers]`, `[workers]`).

Tâche une action unitaire, exprimée par un *module* : `apt`, `file`, `template`, `systemd`.

Rôle un ensemble de tâches réutilisable, avec ses variables par défaut, ses fichiers et ses gabarits. Le projet en compte neuf.

Playbook la partition : quels rôles sur quels groupes, dans quel ordre.

Handler une tâche qui ne s'exécute que si une autre a changé quelque chose. Typiquement : redémarrer un service.

Vault un fichier chiffré où ranger les secrets, versionnable sans danger.

CHAPITRE 5

DevSecOps : pourquoi une chaîne automatique

5.1 Le constat

Un projet d'école finit souvent par ne plus fonctionner que sur la machine de son auteur, parce que rien ne vérifie le contraire. L'intégration continue consiste à faire tourner, à chaque envoi de code, l'ensemble des contrôles que personne n'a le courage de relancer à la main.

Le « Sec » de DevSecOps signifie que la sécurité fait partie de ces contrôles automatiques, plutôt que d'être un audit ponctuel en fin de projet.

5.2 Les familles de contrôles

Lint lit le code sans l'exécuter et signale ce qui est douteux : variable jamais utilisée, import mort, nom inconnu. Outils : `ruff` pour Python, `hadolint` pour les Dockerfiles, `ansible-lint`.

Tests exécutent le code sur des cas connus et comparent au résultat attendu. Outil : `pytest`.

SAST *Static Application Security Testing* — cherche des motifs dangereux dans le source : concaténation SQL, secret en dur, appel système construit à partir d'une entrée utilisateur. Outil : `bandit`.

SCA *Software Composition Analysis* — confronte la liste des dépendances aux bases de vulnérabilités publiques. Outil : `pip-audit`.

Recherche de secrets cherche clés et mots de passe commis par erreur, *y compris dans l'historique* : effacer un secret dans un commit ultérieur ne l'efface pas du dépôt. Outil : `gitleaks`.

DAST *Dynamic Application Security Testing* — attaque l'application *en fonctionnement* : en-têtes manquants, pages accessibles sans authentification, jetons CSRF absents. Outils : un script maison et `nuclei`.

Scan d'image cherche les CVE des paquets du système de base de l'image. Outil : `Trivy`.

SBOM *Software Bill of Materials* — l'inventaire exhaustif de ce qui est livré, conservé pour pouvoir répondre plus tard à « sommes-nous touchés par telle faille ? ». Outil : `Syft`.

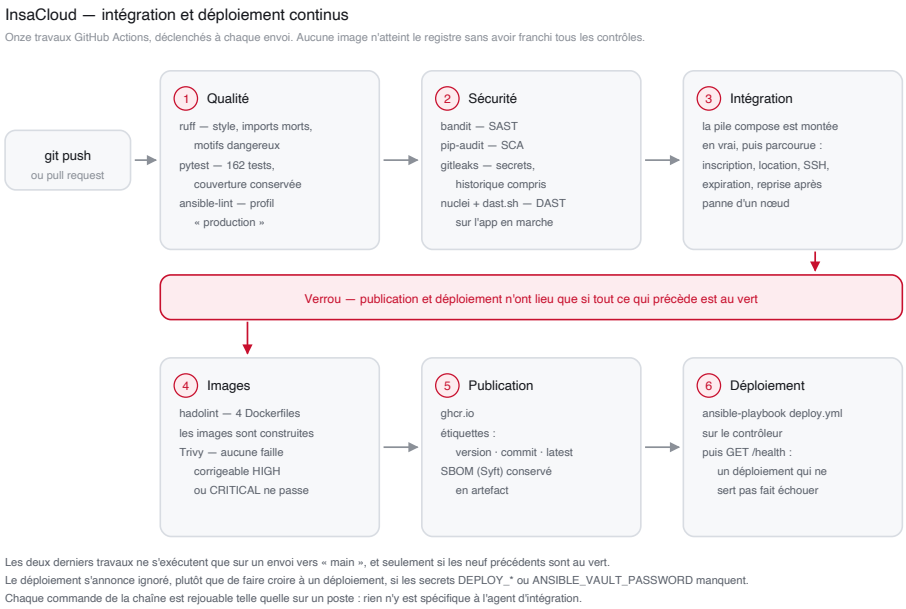


Figure 5.1 – La chaîne réelle du projet : onze travaux, détaillés au chapitre 23.

Notion — CVE

Common Vulnerabilities and Exposures — l'identifiant mondial unique d'une faille connue, de la forme `CVE-2024-12345`. Les outils de SCA et de scan d'image travaillent tous avec ces identifiants.

À retenir

Une chaîne d'intégration n'est utile que si elle *bloque*. Un contrôle qui affiche un avertissement que personne ne lit ne sert à rien. Ici, aucune image n'atteint le registre si l'un des contrôles échoue — et c'est arrivé : nos propres images ont été refusées par notre propre scan (chapitre 23).

Deuxième partie

L'architecture

CHAPITRE 6

La carte du territoire

6.1 Un contrôleur, trois exécutants

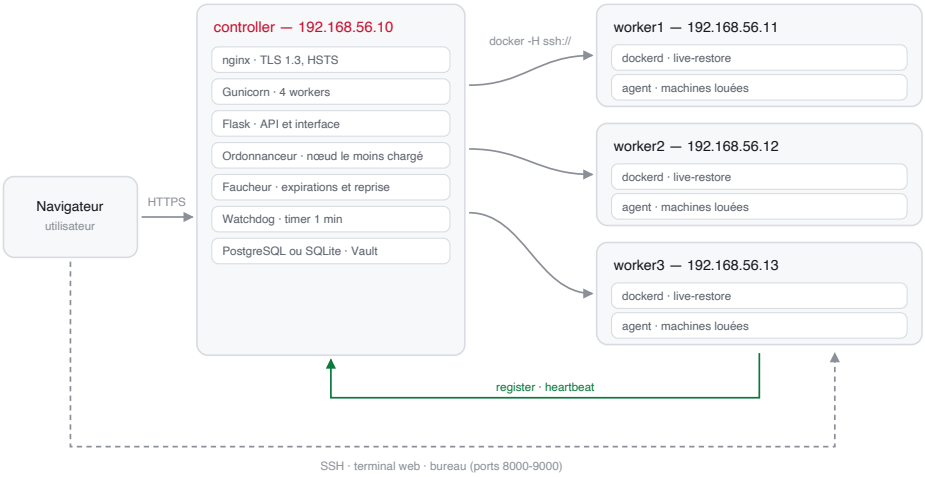


Figure 6.1 – L'architecture déployée. Le contrôleur décide, les workers exécutent.

La plateforme se répartit sur quatre machines.

Le contrôleur (192.168.56.10) porte tout ce qui réfléchit : le site web, l'API, la base de données, l'ordonnanceur qui choisit où placer une machine, et le Faucheur qui détruit celles qui ont expiré. Il n'exécute *aucune* machine louée et n'a même pas de démon Docker — seulement le client.

Les workers (.11, .12, .13) portent tout ce qui exécute : un démon Docker, les conteneurs loués, et un petit agent qui signale leur présence.

> **À retenir**
>
> Cette séparation n'est pas décorative. Elle garantit qu'un utilisateur qui s'échapperait de son conteneur se retrouverait sur un worker — une machine sans base de données, sans clés, sans code de la plateforme. Le contrôleur, lui, n'exécute rien qui vienne de l'extérieur.

6.2 Comment le contrôleur pilote les workers

Il existe deux façons de commander un démon Docker distant.

Exposer l'API Docker sur le réseau (port 2375/2376). C'est simple, et c'est une très mauvaise idée : qui parle au démon Docker d'une machine en est root. Une erreur de pare-feu et toute la plateforme est compromise.

Passer par SSH. Le client Docker sait le faire nativement :

```
docker -H ssh://insacloud@worker1 ps
```

Aucun port Docker n'est ouvert. L'authentification est celle de SSH, par clé. Le compte `insacloud` du worker n'a pas de mot de passe et n'est joignable que depuis le contrôleur.

Listing 6.1 — `2_webapp/workers.py` — le préfixe de commande

```
def docker_command(worker: str = "local") -> list:
    """Prefixe de commande docker pour ce noeud."""
    host = worker_host(worker)
    if host is None:
        return ["docker"]
    return ["docker", "-H", f"ssh://{DOCKER_SSH_USER}@{host}"]
```

Toute commande Docker du projet passe par cette fonction. Conséquence heureuse : le code est le même en mono-hôte (développement) et en cluster (production). Seul le préfixe change.

> **Notion — Le mode mono-hôte**
>
> Si l'inventaire ne déclare aucun worker, `worker_host` renvoie `None`, le préfixe se réduit à `docker`, et tout tourne sur la machine locale. C'est ce qui permet de développer sur un portable sans monter quatre VM, et c'est ce que fait la pile `docker compose`.

6.3 Comment le contrôleur apprend l'existence des workers

C'est le point qui distingue une maquette d'une vraie plateforme : **rien n'est écrit en dur**. Le contrôleur ne contient aucune liste d'adresses de workers.

Chaque worker porte un petit agent qui, au démarrage, se présente au contrôleur, puis signale régulièrement qu'il est toujours vivant.

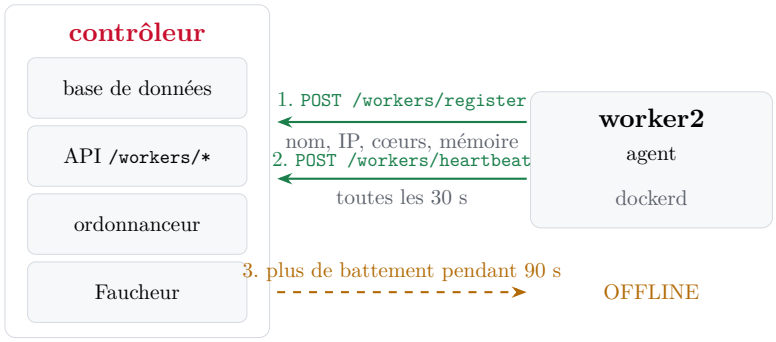


Figure 6.2 – L'inscription spontanée. Allumer un worker suffit à l'ajouter au parc ; l'éteindre suffit à l'en sortir.

> **À retenir**
>
> Ajouter un quatrième worker ne demande de modifier aucun fichier sur le contrôleur. On l'allume, il s'inscrit, l'ordonnanceur commence à lui envoyer des machines. C'est la **Fonctionnalité 7** du cahier des charges, et c'est ce qui rend le parc élastique.

CHAPITRE 7

Le modèle de données

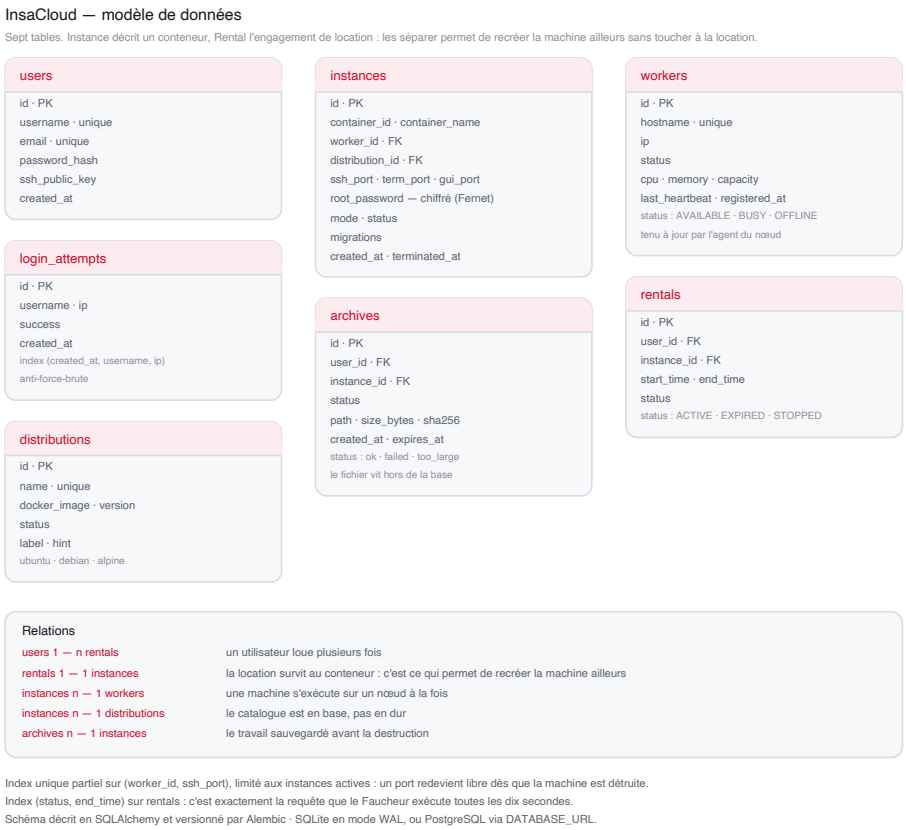


Figure 7.1 – Les sept tables. La séparation `instances` / `rentals` est la décision qui structure tout le reste.

7.1 Pourquoi séparer la machine de la location

C'est le choix le moins évident et le plus important du projet.

On pourrait n'avoir qu'une table : une ligne par location, contenant à la fois l'engagement (quel utilisateur, jusqu'à quand) et la machine (quel conteneur, quel port, sur quel nœud). C'est plus court, et c'est ce qu'on écrit naturellement.

Cela rend la reprise après panne **impossible**.

Quand un nœud tombe, il faut recréer la machine ailleurs : nouveau conteneur, nouveau port, nouveau nœud, nouveau mot de passe. Mais la location, elle, ne doit pas bouger : l'utilisateur a réservé trente minutes, il doit garder ses trente minutes. Avec une table unique, on ne peut pas changer l'un sans toucher à l'autre.

`Instance` décrit un conteneur : son identifiant Docker, son nœud, ses ports, son mot de passe chiffré, son état.

`Rental` décrit l'engagement : quel utilisateur, de quand à quand, actif ou non.

La reprise change l'`Instance` et ne touche pas au `Rental`.

> **À retenir**
>
> Quand une table mélange « ce qui est promis » et « ce qui l'exécute », toute panne devient visible par l'utilisateur. Séparer les deux est ce qui permet de réparer sans rien lui reprendre.

7.2 Les sept tables

Table	Ce qu'elle porte
`users`	comptes : identifiant, empreinte du mot de passe, clé SSH publique
`login_attempts`	tentatives de connexion, pour le verrouillage anti-force-brute
`distributions`	le catalogue : Ubuntu, Debian, Alpine — en base, pas en dur
`workers`	le parc : nom, adresse, état, charge, dernier battement de cœur
`instances`	les conteneurs : identifiant Docker, nœud, ports, secret chiffré
`rentals`	les locations : utilisateur, début, fin, état
`archives`	le travail sauvegardé avant destruction

7.3 Deux index qui portent une règle métier

Un index n'est pas qu'une optimisation : il peut imposer une contrainte que le code aurait du mal à garantir seul.

Listing 7.1 — `2_webapp/models.py` — l'unicité du port

```
__table_args__ = (
    Index("idx_instances_status", "status"),
    # Index unique PARTIEL : deux machines actives ne peuvent pas
        ↪ partager un
    # port sur le meme noeud. La clause « where » le limite aux
        ↪ instances en
    # cours, donc un port redevient libre des que la machine est
        ↪ detruite.
    Index("uq_worker_port_actif", "worker_id", "ssh_port", unique=True,
```

```
            sqlite_where=text("status = 'running'"),
            postgresql_where=text("status = 'running'")),
)
```

Notion — Index unique partiel

Un index unique classique interdirait deux fois le même port *pour toujours* : les machines détruites continueraient de réserver le leur. La clause `WHERE` restreint la contrainte aux lignes qui comptent.
Sans cela, deux requêtes simultanées pourraient choisir le même port libre et créer deux conteneurs en conflit — une *situation de compétition* que le code applicatif ne peut pas écarter seul, car entre « ce port est libre » et « je le prends » il s'écoule toujours un instant.

Le second index sert le Faucheur :

```
__table_args__ = (Index("idx_rentals_end", "status", "end_time"),)
```

C'est exactement la requête qu'il exécute toutes les dix secondes : « les locations actives dont la date de fin est passée ». Sans index, chaque passage relirait toute la table.

CHAPITRE 8

Le cycle de vie d'une machine

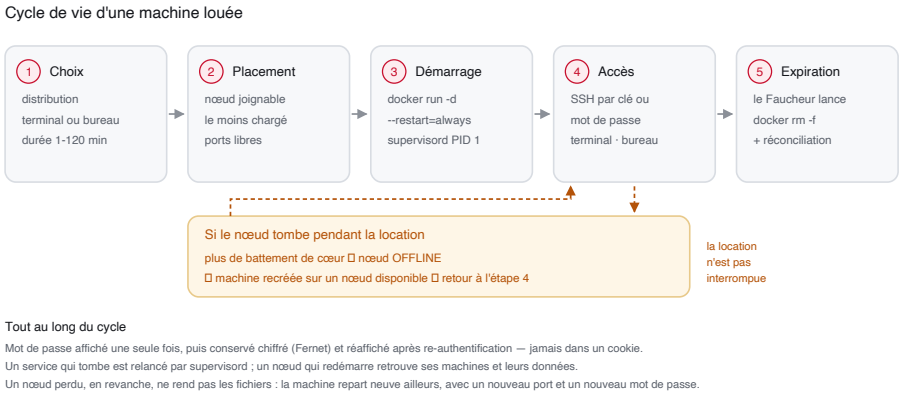


Figure 8.1 – De la demande à l'expiration, avec la branche de reprise si un nœud tombe pendant la location.

8.1 Les cinq étapes

1. **Choix.** L'utilisateur sélectionne une distribution, un mode (terminal ou bureau) et une durée de 1 à 120 minutes. Les trois valeurs sont validées côté serveur — ce qui vient du navigateur n'est jamais cru.

2. **Placement.** L'ordonnanceur retient le nœud joignable le moins chargé, puis réserve un port libre dans la plage 8000–9000.
3. **Démarrage.** `docker run -d --restart=always -p <port>:22` crée le conteneur ; `supervisord` y démarre en PID 1 et lance les services du mode choisi.
4. **Accès.** L'utilisateur reçoit une commande SSH et, selon le mode, l'adresse d'un terminal web ou d'un bureau. Le mot de passe root est affiché *une seule fois*.
5. **Expiration.** Le Faucheur détruit la machine à l'échéance et libère le port.

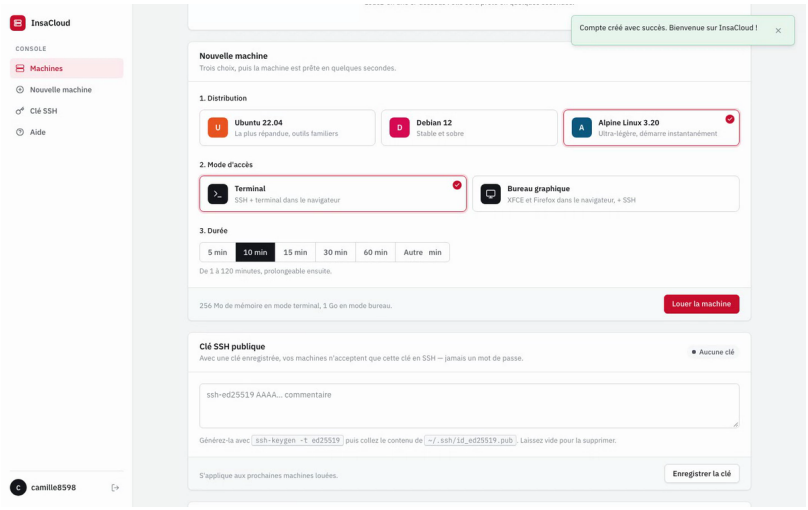


Figure 8.2 – L'étape 1 vue par l'utilisateur. Une distribution dont l'image n'est pas construite sur le parc n'apparaît pas : le catalogue interroge Docker avant d'afficher un choix.

8.2 La branche qui fait la différence

Si le nœud tombe *pendant* la location, l'agent cesse de battre. Au bout de 90 secondes sans nouvelle, le contrôleur déclare le nœud hors ligne et recrée ses machines sur un nœud disponible. L'utilisateur garde sa location, sa durée et son droit d'accès ; il reçoit un nouveau port et un nouveau mot de passe.

> **Piège — La reprise ne rend pas les fichiers**
>
> Il faut le dire clairement, parce que c'est une limite assumée et pas un oubli : les fichiers créés dans l'ancienne machine vivaient dans sa couche inscriptible, sur un nœud devenu injoignable. Sans stockage partagé entre les nœuds, ils sont perdus. Seul le *service* repart.
> Un volume répliqué (NFS, Ceph) changerait la nature du projet et son périmètre. C'est un choix documenté, pas une négligence.

8.3 Trois niveaux de résilience

La haute disponibilité se joue à trois échelles emboîtées, et c'est ce qu'attend la **Fonctionnalité 13**.

Échelle	Mécanisme	Ce qu'il rattrape
Dans la machine	`supervisord`	un service meurt (sshd, ttyd, VNC) : il est relancé
Sur le nœud	`--restart=always`	supervisord meurt : Docker relance le conteneur
Sur le nœud	`live-restore`	le démon Docker redémarre : les conteneurs continuent
Dans le parc	battement de cœur	le nœud entier tombe : les machines repartent ailleurs

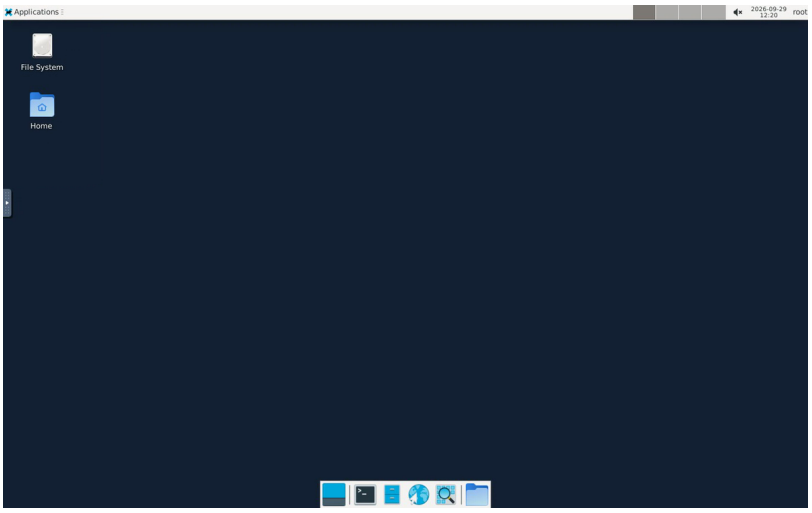


Figure 8.3 – Deux machines en cours, une par mode d'accès. Le temps restant est calculé à l'affichage ; c'est le Faucheur, et non la page, qui décide de la destruction.

> **Notion — `live-restore`**
>
> Par défaut, redémarrer le démon Docker arrête tous ses conteneurs. L'option `live-restore` les laisse tourner pendant ce redémarrage. Pour une plateforme de location, c'est la différence entre « une mise à jour de Docker coupe tous les clients » et « personne ne s'en aperçoit ».

Troisième partie

La construction

CHAPITRE 9

Les images des machines louées

9.1 Le problème : plusieurs services dans un conteneur

Un conteneur exécute *un* processus. Quand ce processus se termine, le conteneur s'arrête. C'est voulu : un conteneur est censé faire une chose.

Or une machine louée doit en faire plusieurs à la fois : un serveur SSH, un terminal web, et en mode bureau un serveur d'affichage, un gestionnaire de fenêtres et une passerelle VNC vers le navigateur. Cinq processus.

Il faut donc un *superviseur* : un processus qui démarre les autres, les surveille, et relance celui qui meurt. Nous utilisons `supervisord`.

> **Notion — PID 1 et la responsabilité d'*init***
>
> Le premier processus d'un espace de noms PID porte le numéro 1 et hérite d'une obligation : adopter les processus orphelins et récupérer leur code de sortie. Sans cela, les processus terminés s'accumulent à l'état *zombie*.
>
> Un `bash` lancé comme PID 1 ne fait pas ce travail. `supervisord`, si.

9.2 Trois Dockerfiles, six images

Le projet propose trois distributions en deux modes. Plutôt que six fichiers, on en écrit trois — un par distribution — et le mode devient un paramètre de construction.

Listing 9.1 — `1_docker/Dockerfile` — la structure (extraits)

```
FROM ubuntu:22.04

# DESKTOP=1 ajoute le serveur d'affichage, XFCE et noVNC.
ARG DESKTOP=0

RUN apt-get update \
    && apt-get install -y --no-install-recommends \
         openssh-server supervisor ca-certificates \
    && rm -rf /var/lib/apt/lists/*

# ... installation conditionnelle du bureau selon DESKTOP ...
```

```
COPY entrypoint.sh /usr/local/bin/entrypoint.sh
RUN chmod +x /usr/local/bin/entrypoint.sh

EXPOSE 22 7681 6080
CMD ["/usr/local/bin/entrypoint.sh"]
```

Piège — `apt-get install` sans nettoyage dans le même `RUN`

Chaque instruction `RUN` crée une couche. Si l'on installe des paquets dans une couche et qu'on efface le cache d'apt dans la couche *suivante*, le cache reste présent dans l'image : une couche ne peut pas alléger celle du dessous.
Il faut donc enchaîner installation et nettoyage dans *le même* `RUN`, comme ci-dessus. Sur nos images, cela représente une cinquantaine de mégaoctets.

Piège — Un mot de passe par défaut dans l'image

Notre première version contenait `ENV ROOT_PASSWORD=insacloud`. L'analyseur `hadolint` l'a signalé (règle DL3064), et il avait raison : ce mot de passe aurait été le même sur *toutes* les machines louées, et visible par quiconque fait `docker history` sur l'image.
La correction : aucun mot de passe dans l'image. Si la plateforme n'en fournit pas, l'entrypoint en tire un au hasard.

9.3 Le script de démarrage

Listing 9.2 – `1_docker/entrypoint.sh` — le repli sûr

```
# Mot de passe root : fourni par la plateforme (-e ROOT_PASSWORD=...).
    ↪ S'il
# manque, on en tire un au hasard plutot que de retomber sur une valeur
    ↪  connue :
# un mot de passe par defaut serait le meme sur toutes les machines
    ↪ louees.
if [ -z "${ROOT_PASSWORD:-}" ]; then
    ROOT_PASSWORD="$(tr -dc 'A-Za-z2-9' < /dev/urandom | head -c 16)"
    echo "[insacloud] Aucun ROOT_PASSWORD fourni : tire au hasard = ${
        ↪ ROOT_PASSWORD}"
fi
```

Le script écrit ensuite une configuration `supervisord` adaptée au mode, puis se remplace par le superviseur :

```
exec "$(command -v supervisord)" -c "$CONF"
```

Notion — Pourquoi `exec`

Sans `exec`, le shell resterait PID 1 et lancerait supervisord comme enfant. Les signaux envoyés par Docker (`docker stop`) iraient au shell, qui ne les relaierait pas : l'arrêt deviendrait brutal au bout du délai de grâce.
`exec` remplace le processus courant : supervisord *devient* le PID 1 et reçoit les signaux directement.

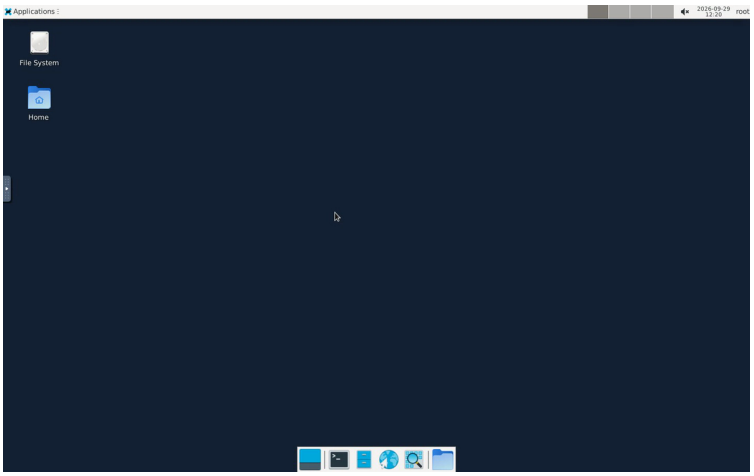


Figure 9.1 – Le mode « bureau » : cinq processus cohabitent dans le conteneur — sshd, le terminal web, le serveur d'affichage, XFCE et la passerelle noVNC — tous surveillés par supervisord.

9.4 Rotation du mot de passe à chaud

L'utilisateur peut régénérer le mot de passe de sa machine sans la redémarrer. Le même script sert de point d'entrée et d'outil :

```
docker exec <conteneur> /usr/local/bin/entrypoint.sh setpass <nouveau>
```

SSH, terminal web et bureau prennent le nouveau mot de passe immédiatement.

CHAPITRE 10

Piloter Docker depuis Python

10.1 Pourquoi la ligne de commande et pas un SDK

Il existe une bibliothèque Python officielle (`docker-py`). Le cahier des charges impose pourtant une commande précise :

```
docker run -d --restart=always -p <port>:22 <image>
```

Piloter la CLI par `subprocess` rend cette commande *visible dans le code*. Chaque option passée se lit telle quelle. C'est un choix de lisibilité et de conformité, pas une facilité.

Listing 10.1 – `2_webapp/docker_ops.py` — l'appel central

```
def run_docker(*args, worker: str = "local", timeout: int =
    ↪ DOCKER_TIMEOUT) -> str:
    """Execute `docker <args>` sur le noeud `worker` (local ou distant
        ↪ via SSH)."""
    cmd = [*wk.docker_command(worker), *args]
    try:
        # Appel sans shell : les arguments sont passes en liste, donc
            ↪ jamais
        # re-interpretes par un interpreteur de commandes. Les seules
            ↪ valeurs
        # d'origine utilisateur (duree, distribution, mode) sont
            ↪ validees en amont.
        res = subprocess.run(cmd, capture_output=True, text=True,
            ↪ timeout=timeout)
    except FileNotFoundError as erreur:
        raise DockerError("La commande 'docker' est introuvable sur le
            ↪ serveur.") from erreur
    except subprocess.TimeoutExpired as erreur:
        raise DockerError("Docker n'a pas repondu dans le delai imparti
            ↪ .") from erreur
    if res.returncode != 0:
        message = res.stderr.strip() or f"docker {args[0]} a echoue ({
            ↪ res.returncode})"
        raise DockerError(message)
    return res.stdout.strip()
```

Piège — `shell=True` est une porte ouverte

Écrire `subprocess.run(f"docker run ... {nom}", shell=True)` laisserait un utilisateur nommer sa machine `x; rm -rf /` et faire exécuter n'importe quoi au serveur. C'est une *injection de commande*.
Passer une **liste** d'arguments, sans `shell`, supprime le problème à la racine : il n'y a aucun interpréteur pour relire la chaîne. Chaque élément de la liste est un argument, point.

10.2 La création d'une machine

Listing 10.2 – `2_webapp/docker_ops.py` — la commande imposée, complétée

```
def docker_run_container(port, name, root_password, os_type, mode,
                         term_port, gui_port=None, ssh_public_key=None,
                         worker="local") -> str:
    spec = MODES[mode]
    args = [
        "run", "-d",
        "--restart=always",
        "-p", f"{port}:22",
        "-p", f"{term_port}:{TERM_CONTAINER_PORT}",
        "--name", name,
        "-e", f"ROOT_PASSWORD={root_password}",
        *CONTAINER_HARDENING,
    ]
    if ssh_public_key:
        args += ["-e", f"SSH_PUBKEY={ssh_public_key}"]
    if spec["gui"] and gui_port:
        args += ["-p", f"{gui_port}:{GUI_CONTAINER_PORT}", "--shm-size"
            ↪ , "512m"]
    if spec["memory"]:
        # Le cours impose de definir reservation ET limite : la
            ↪ reservation est
        # le minimum garanti en cas de contention, la limite le plafond
            ↪  dur.
        args += ["--memory", spec["memory"],
                 "--memory-reservation", spec["reservation"]]
    args.append(image_for(os_type, mode))
    return run_docker(*args, worker=worker)[:12]
```

10.3 Le durcissement, option par option

```
CONTAINER_HARDENING = [
    "--label", f"{MACHINE_LABEL}={MACHINE_LABEL_VALUE}",
    "--label", f"{CLUSTER_LABEL}={CLUSTER}",
    "--cap-drop", "ALL",
```

```
    "--cap-add", "CHOWN", "--cap-add", "DAC_OVERRIDE", "--cap-add", "
        ↪ FOWNER",
    "--cap-add", "FSETID", "--cap-add", "SETGID", "--cap-add", "SETUID"
        ↪ ,
    "--cap-add", "SYS_CHROOT", "--cap-add", "NET_BIND_SERVICE",
    "--cap-add", "KILL", "--cap-add", "AUDIT_WRITE",
    "--security-opt", "no-new-privileges:true",
    "--pids-limit", CONTAINER_PIDS_LIMIT,
    "--cpus", CONTAINER_CPUS,
]
```

Option	Ce qu'elle empêche
`--cap-drop ALL`	tout pouvoir de root, avant d'en rendre dix
`--cap-add NET_BIND_SERVICE`	sans elle, sshd ne peut pas écouter sur le port 22
`no-new-privileges`	qu'un binaire *setuid* regagne des droits
`--pids-limit`	une *fork bomb* qui sature la table des processus
`--cpus`	qu'un minage monopolise les cœurs du nœud
`--memory`	qu'une fuite mémoire fasse tomber les voisins

10.4 Reconnaître ses machines : l'erreur qui a tout détruit

Le Faucheur doit savoir quels conteneurs lui appartiennent. Notre première version les reconnaissait à leur nom, qui commence par `insacloud_`.

Puis nous avons déployé la pile `docker compose`, dont les conteneurs s'appellent `insacloud-web`, `insacloud-db`, `insacloud-faucheur`. Le Faucheur les a pris pour des machines louées orphelines et les a détruits. **Il s'est supprimé lui-même.**

Listing 10.3 – `2_webapp/docker_ops.py` — l'identité par étiquette

```
# Label appose a TOUTE machine louee. C'est lui, et non le nom du
    ↪ conteneur,
# qui identifie ce que la plateforme a le droit de detruire : un
    ↪ prefixe de nom
# est trop facile a percuter.
MACHINE_LABEL = "insacloud.role"
MACHINE_LABEL_VALUE = "rented-machine"
CLUSTER_LABEL = "insacloud.cluster"

# Les deux filtres sont combines : une machine doit etre une machine
    ↪ louee ET
# appartenir a CE deploiement. Sans le second, le Faucheur d'une
    ↪ plateforme
# detruirait les machines d'une autre tournant sur le meme demon.
MACHINE_FILTERS = [f"label={MACHINE_LABEL}={MACHINE_LABEL_VALUE}",
                   f"label={CLUSTER_LABEL}={CLUSTER}"]
```

À retenir

Un nom est une commodité d'affichage ; une étiquette est une identité. Dès qu'un programme doit décider de *détruire* quelque chose, il doit se fonder sur une marque qu'il a lui-même apposée, jamais sur une convention de nommage.

Le second label a été ajouté après un deuxième incident : deux déploiements partageant un même démon Docker se détruisaient mutuellement leurs conteneurs.

CHAPITRE 11

La base de données

11.1 SQLAlchemy : pourquoi un ORM

On peut écrire du SQL à la main. On s'expose alors à trois choses : les injections SQL si l'on concatène des chaînes, l'incompatibilité entre SQLite et PostgreSQL, et l'absence de vérification à l'écriture.

Un *ORM* (*Object-Relational Mapper*) fait correspondre une classe Python à une table. On écrit des objets, il écrit le SQL — toujours avec des paramètres liés, donc sans injection possible.

Listing 11.1 – `2_webapp/models.py` — une table

```
class Rental(Base):
    __tablename__ = "rentals"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"),
        ↪ nullable=False)
    instance_id: Mapped[int] = mapped_column(ForeignKey("instances.id")
        ↪ , nullable=False)
    start_time: Mapped[datetime] = mapped_column(DateTime, default=
        ↪ utc_now, nullable=False)
    end_time: Mapped[datetime] = mapped_column(DateTime, nullable=False
        ↪ )
    status: Mapped[str] = mapped_column(String(16), default=
        ↪ RENTAL_ACTIVE, nullable=False)

    user: Mapped["User"] = relationship(back_populates="rentals")
    instance: Mapped["Instance"] = relationship(back_populates="rental"
        ↪ )

    __table_args__ = (Index("idx_rentals_end", "status", "end_time"),)
```

Notion — Clé étrangère

`ForeignKey("users.id")` déclare que cette colonne pointe vers la table `users`. La base refusera alors une location dont l'utilisateur n'existe pas : la cohérence est garantie par le moteur, pas par la discipline du développeur.

11.2 Alembic : faire évoluer un schéma sans perdre les données

Tant qu'on développe, on efface la base et on recommence. Dès qu'il y a des données réelles, c'est impossible : il faut *migrer*.

Alembic enregistre chaque changement de schéma dans un fichier numéroté, et chaque fichier connaît son prédécesseur. La base retient où elle en est.

```
# Comparer les modeles a la base et generer la migration correspondante
alembic revision --autogenerate -m "archives du travail des
    ↪ utilisateurs"

# Appliquer tout ce qui manque
alembic upgrade head

# Verifier que la base correspond aux modeles (utilise par la CI)
alembic check
```

Piège — SQLite ne sait pas modifier une colonne

PostgreSQL accepte `ALTER TABLE ... ALTER COLUMN`. SQLite, non : il faut créer une table neuve, recopier, supprimer l'ancienne, renommer. Alembic sait le faire, mais seulement si on le lui demande avec `render_as_batch=True`. Sans cette option, les migrations passent en développement sur PostgreSQL et échouent sur le SQLite d'un poste — ou l'inverse.

11.3 La course au démarrage : un bug qui ne se voit qu'en production

Gunicorn lance quatre processus. Chacun importe l'application, qui appelle `init_db()`, qui crée les tables si elles manquent. Les quatre le font *en même temps*.

Le résultat est intermittent : parfois tout se passe bien, parfois un processus meurt avec `table users already exists`. Sur un poste de développement mono-processus, le bug n'apparaît jamais.

Listing 11.2 — `2_webapp/database.py` — création tolérante à la concurrence

```
for tentative in range(5):
    try:
        Base.metadata.create_all(engine)
        break
    except (OperationalError, ProgrammingError) as erreur:
        message = str(erreur).lower()
```

```
        if "already exists" in message:
            # Un autre processus a cree le schema entre notre
                ↪ verification et
            # notre creation : c'est exactement le resultat voulu.
            log.info("Schema cree simultanement par un autre processus.
                ↪ ")
            break
        if "locked" not in message or tentative == 4:
            raise
        time.sleep(0.3 * (tentative + 1))
```

Nous avons reproduit la panne en lançant huit processus simultanés : six sur huit démarraient. Après correction, huit sur huit. La chaîne d'intégration lance désormais Gunicorn avec `--preload`, qui importe l'application *une fois* avant de dupliquer le processus.

> **À retenir**
>
> Un bug de concurrence ne se trouve pas en relisant le code : il se trouve en lançant plusieurs processus à la fois et en recommençant. Si vous ne savez pas reproduire un bug, vous ne saurez pas vérifier qu'il est corrigé.

11.4 Ne jamais journaliser un secret

Le Faucheur affichait au démarrage l'URL de la base — qui contient le mot de passe PostgreSQL — en clair dans `journalctl`. Trouvé en lisant nos propres journaux. Toute URL passe désormais par une fonction de masquage avant d'être écrite.

Listing 11.3 – `2_webapp/database.py`

```
def url_sans_secret(url: str) -> str:
    """Masque le mot de passe d'une URL de connexion avant de la
        ↪ journaliser."""
```

CHAPITRE 12

L'application et son API

12.1 Une organisation en couches

Fichier	Responsabilité
app.py	les routes HTML, les sessions, les en-têtes de sécurité
api.py	les routes JSON du cahier des charges
services.py	la logique métier : louer, arrêter, prolonger, reprendre
database.py	tous les accès à la base
docker_ops.py	tout ce qui parle à Docker
models.py	la description des tables
crypto.py	les clés et le chiffrement
workers.py	la résolution des nœuds et le choix du moins chargé

La règle qui tient l'ensemble : **les couches basses ne connaissent pas les hautes**. docker_ops.py ignore l'existence de Flask et de la base. On peut donc le tester sans serveur, et le Faucheur peut l'utiliser directement.

12.2 Le résultat d'une opération

Plutôt que de renvoyer des exceptions ou des tuples, les services renvoient un objet uniforme :

Listing 12.1 — 2_webapp/services.py

```
@dataclass
class Resultat:
    ok: bool
    message: str
    code: int = 200
    donnees: dict = field(default_factory=dict)
```

La route HTML en tire un message `flash` et une redirection ; la route JSON en tire un corps et un code HTTP. La logique métier, elle, ne sait pas laquelle des deux l'appelle.

12.3 Les points d'entrée imposés

Le cahier des charges fixe un contrat d'API précis. Les voici, tels qu'implémentés :

Méthode	Chemin	Rôle
GET	`/health`	état du service, de la base et du parc
GET	`/distributions`	le catalogue des systèmes disponibles
GET	`/instances`	les machines de l'utilisateur connecté
GET	`/instances/{id}`	le détail d'une machine
POST	`/rent`	louer une machine
POST	`/instances/{id}/stop`	arrêter et détruire
POST	`/instances/{id}/extend`	prolonger la location
GET	`/workers`	l'état du parc
GET	`/workers/{id}`	le détail d'un nœud
POST	`/workers/register`	inscription d'un agent
POST	`/workers/heartbeat`	battement de cœur d'un agent

Listing 12.2 – Louer une machine en JSON

```
curl -s -c biscuits -b biscuits -H 'Content-Type: application/json' \
     -d '{"username":"aziz","password":"..."}' http://127.0.0.1:8088/
         ↪ login

curl -s -c biscuits -b biscuits -H 'Content-Type: application/json' \
     -d '{"distribution":"alpine","mode":"terminal","duration":30}' \
     http://127.0.0.1:8088/rent | jq
```

Notion — Pourquoi `/health` compte

Une sonde d'état permet à un orchestrateur, à un équilibreur de charge ou à une chaîne de déploiement de savoir si le service est réellement utilisable. Ici, elle vérifie aussi la base et le parc : un service qui répond mais dont la base est injoignable n'est pas en bonne santé.
La chaîne d'intégration s'en sert comme preuve finale : un déploiement qui ne sert pas fait échouer la chaîne.

12.4 Protéger les routes des agents

Les agents ne sont pas des utilisateurs : ils n'ont pas de session. Ils s'authentifient par un jeton partagé, dans un en-tête.

Listing 12.3 – `2_webapp/api.py` — comparaison en temps constant

```
def token_agent_requis(vue):
    @functools.wraps(vue)
    def enveloppe(*args, **kwargs):
        presente = request.headers.get("X-Agent-Token", "")
        # compare_digest : la duree de la comparaison ne depend pas du
            ↪ nombre
        # de caracteres corrects. Un « == » permettrait de deviner le
            ↪ jeton
```

```
        # caractere par caractere en mesurant le temps de reponse.
        if not AGENT_TOKEN or not hmac.compare_digest(presente,
            ↪ AGENT_TOKEN):
            return jsonify(error="Jeton d'agent invalide."), 401
        return vue(*args, **kwargs)
    return enveloppe
```

Notion — Attaque temporelle

L'opérateur `==` sur deux chaînes s'arrête au premier caractère différent. En mesurant très précisément le temps de réponse, un attaquant peut découvrir le secret caractère par caractère. `hmac.compare_digest` compare toujours l'intégralité, en temps constant.

CHAPITRE 13

Le parc de nœuds

13.1 L'agent, en bibliothèque standard seulement

L'agent tourne sur chaque worker. Il pourrait utiliser `requests`, mais cela imposerait d'installer des dépendances Python sur chaque nœud. Il n'emploie donc que la bibliothèque standard : `urllib` pour les appels, `/proc/meminfo` pour la mémoire, `os.cpu_count()` pour les cœurs.

Listing 13.1 — `2_webapp/worker_agent.py` — se présenter

```
def enregistrer() -> bool:
    """Presente le noeud au controleur. Retourne True si l'inscription
        ↪ a pris."""
    charge = {
        "hostname": nom_du_noeud(),
        "ip": adresse_du_noeud(),
        "cpu": nombre_de_coeurs(),
        "memory": memoire_totale_mio(),
        "docker": docker_operationnel(),
    }
    code, corps = appeler("/workers/register", charge)
    if code in (200, 201):
        log.info("Enregistre aupres de %s : %s (%s coeurs, %s Mio,
            ↪ capacite %s)",
                 CONTROLLER, charge["hostname"], charge["cpu"], charge[
                     ↪ "memory"],
                 corps.get("capacity"))
        return True
    log.error("Enregistrement refuse (%s) : %s", code, corps.get("error
        ↪ ", corps))
    return False
```

13.2 Le battement de cœur, et les trois états

Listing 13.2 — `2_webapp/worker_agent.py` — signaler sa présence

```
def battre() -> bool:
    """
    Envoie un battement de coeur. Annonce BUSY si le noeud est sature,
        ↪ pour que
    l'ordonnanceur cesse d'y envoyer des machines sans le declarer en
        ↪ panne.
    """
```

```
    if not docker_operationnel():
        log.warning("Docker ne repond pas : aucun battement envoye.")
        return False
    charge = {"hostname": nom_du_noeud(), "containers":
        ↪ conteneurs_actifs()}
    code, corps = appeler("/workers/heartbeat", charge)
    if code == 200:
        return True
    if code == 404 and corps.get("action") == "register":
        log.warning("Noeud inconnu du controleur : nouvelle inscription
            ↪ .")
        return enregistrer()
    return False
```

État	Signification
`AVAILABLE`	le nœud bat, Docker répond, il reste de la place
`BUSY`	le nœud bat, mais il a atteint sa capacité : on ne lui envoie plus rien
`OFFLINE`	plus de battement depuis 90 s : ses machines sont recréées ailleurs

À retenir

La distinction `BUSY` / `OFFLINE` est essentielle. Un nœud saturé est en bonne santé : il ne faut surtout pas détruire et recréer ses machines ailleurs. Confondre « plein » et « mort » déclencherait des migrations inutiles et coûteuses.

13.3 Démarrer avant le contrôleur

Au démarrage d'un cluster, les workers peuvent être prêts avant le contrôleur. L'agent ne doit alors ni abandonner, ni marteler le réseau.

```
    # L'inscription est retentee jusqu'a ce qu'elle passe : au
        ↪ demarrage d'un
    # cluster, le worker peut etre pret avant le controleur.
    attente = 5
    while not _arret.is_set() and not enregistrer():
        log.info("Nouvelle tentative dans %s s.", attente)
        _arret.wait(attente)
        attente = min(attente * 2, 60)
```

Notion — Recul exponentiel (*exponential backoff*)

On double le délai à chaque échec, jusqu'à un plafond. Un service qui retenterait toutes les secondes ajouterait sa charge à celle d'un contrôleur déjà en difficulté. Doubler l'attente laisse au système le temps de se rétablir.

13.4 L'ordonnanceur

Le choix du nœud suit deux règles simples et une exclusion :

1. n'envisager que les nœuds `AVAILABLE` — ni saturés, ni muets ;
2. parmi eux, prendre celui qui héberge le moins de machines ;
3. en cas d'égalité, le premier par ordre alphabétique, pour que le comportement soit reproductible et donc testable.

> **À retenir**
>
> Un ordonnanceur déterministe est un ordonnanceur qu'on peut tester. Si le départage en cas d'égalité est aléatoire, le test devient intermittent — et un test intermittent finit toujours par être ignoré.

CHAPITRE 14

Le Faucheur

14.1 Un démon, trois missions

Le Faucheur est un processus séparé de l'application. Il tourne en boucle, toutes les dix secondes.

Expirer détruire les machines dont la location est échue.

Réconcilier remettre Docker et la base d'accord.

Surveiller le parc déclarer hors ligne les nœuds muets et faire recréer leurs machines.

> **Notion — Pourquoi un processus séparé**
>
> On pourrait lancer un fil d'exécution dans l'application. Mauvaise idée : Gunicorn lance quatre processus, donc quatre Faucheurs qui se marcheraient dessus. Un service distinct, démarré une seule fois par systemd, évite le problème et peut être redémarré sans couper le site.

14.2 La réconciliation, ou l'art de douter de soi

La base dit ce qui *devrait* exister ; Docker dit ce qui existe *vraiment*. Les deux divergent : un conteneur tué à la main, un processus interrompu au mauvais moment, un nœud redémarré.

Listing 14.1 — `2_webapp/faucheur.py`

```
def reconcile() -> None:
    """
    Remet Docker et la BDD en coherence, noeud par noeud :
      - conteneur insacloud_* sans instance 'running' en BDD  -> docker
          ↪  rm -f
      - instance 'running' en BDD sans conteneur dans Docker  -> status
          ↪  'stopped'
    Un noeud injoignable est simplement ignore (rien n'est modifie pour
        ↪  lui).
    """
```

> **Piège — Un nœud injoignable n'est pas un nœud vide**
>
> C'est le piège le plus dangereux de la réconciliation. Si le nœud ne répond pas, `docker ps` renvoie une liste vide. Un code naïf en conclurait

que toutes ses machines ont disparu et les marquerait « arrêtées » — alors qu'elles tournent parfaitement.
D'où la règle : un nœud injoignable est *ignoré*, jamais interprété.

14.3 L'erreur du `+-1 minutes`

Prolonger une location ajoute des minutes ; l'écourter en retire. Le code construisait l'intervalle ainsi :

```
f"+{int(minutes)} minutes"      # avec minutes = -1 : "+-1 minutes"
```

SQLite rejette `+-1 minutes`. Il faut demander explicitement le signe :

```
f"{int(duration_minutes):+d} minutes"   # donne "+30 minutes" ou "-1
    ↪ minutes"
```

Notion — Le format `:+d`

En Python, `f"{x:+d}"` affiche toujours le signe : `+30` ou `-1`. C'est exactement ce qu'attend la syntaxe des intervalles SQL.

CHAPITRE 15

La sécurité

15.1 Deux clés, jamais la même

Listing 15.1 – 2_webapp/crypto.py

```
"""
Deux cles distinctes, jamais la meme :
  * la cle de session Flask, qui signe les cookies ;
  * la cle du coffre (Fernet), qui chiffre les mots de passe root en
      ↪ base.

Chacune vient de l'environnement - Ansible les injecte depuis son Vault
    ↪  - ou,
a defaut, d'un fichier local en 0600 cree une seule fois. Separer les
    ↪ deux
garantit qu'un vol de la cle de session ne donne pas acces aux machines
    ↪  louees.
"""
```

Notion — Fernet

Fernet est une recette de chiffrement symétrique authentifié fournie par la bibliothèque `cryptography`. Elle combine AES en mode CBC et un HMAC : on ne peut ni lire le contenu sans la clé, ni le modifier sans que cela se voie.
Son intérêt pour un débutant : il n'y a aucun paramètre à choisir, donc aucune occasion de se tromper de mode ou de vecteur d'initialisation.

15.2 Le mot de passe root : montré une fois, conservé chiffré

1. À la création, 16 caractères sont tirés au sort (environ 90 bits d'entropie).
2. Ils sont affichés *une seule fois* à l'utilisateur.
3. Ils sont chiffrés avec Fernet avant d'être écrits en base.
4. Pour les revoir, l'utilisateur ressaisit le mot de passe de *son compte* : le coffre s'ouvre cinq minutes, puis se reverrouille.

Listing 15.2 – 2_webapp/app.py — plus rien de secret dans le cookie

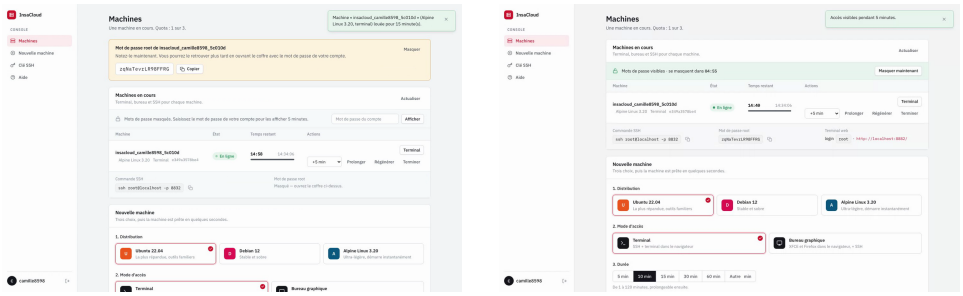


Figure 15.1 – À gauche, l'affichage unique à la création. À droite, le coffre rouvert après re-authentification : les accès redeviennent visibles cinq minutes.

```python
def reveal_secret(instance_id: int, rotated: bool = False) -> None:
    # On ne depose que l'IDENTIFIANT de la machine dans la session. Le
        ↪ secret
    # lui-meme est relu en base, dechiffre, au moment de l'affichage.
    session["reveal"] = {"instance": int(instance_id), "rotated": bool(
        ↪ rotated)}

def secret_a_reveler():
    marqueur = session.pop("reveal", None)
    if not marqueur or g.user is None:
        return None
    # Le proprietaire est verifie dans la requete : l'identifiant d'une
        ↪  machine
    # d'autrui ne donne rien, meme si la session etait forgee.
    instance = db.get_instance(marqueur.get("instance"), user_id=g.user
        ↪ ["id"])
    if instance is None or instance["status"] != db.STATUS_RUNNING:
        return None
    secret = decrypt_secret(instance["root_password"])
```

15.3 CSRF : empêcher un autre site d'agir en votre nom

Notion — *Cross-Site Request Forgery*

Vous êtes connecté à notre plateforme. Vous visitez un site malveillant qui contient un formulaire invisible pointant vers `/instances/3/stop`, soumis automatiquement. Votre navigateur joint vos cookies : la requête est authentifiée, et votre machine s'arrête.

La parade : un jeton imprévisible, déposé dans la session et recopié dans chaque formulaire. Le site malveillant ne peut pas le deviner — il ne peut pas lire votre session.

Piège — Exempter les requêtes JSON, mais pourquoi ?

L'API doit pouvoir être appelée par `curl`, qui n'a pas de jeton. Notre exemption porte sur `Content-Type: application/json`, et elle est sûre pour deux raisons :

— un formulaire HTML ne peut *pas* émettre ce type de contenu — il est limité à trois types, dont aucun n'est JSON ;

— une requête `fetch` qui le fait depuis un autre site déclenche une vérification *preflight* CORS, que le serveur refuse.

Le raisonnement compte autant que la règle : exempter sans comprendre ouvrirait exactement le trou qu'on cherche à boucher.

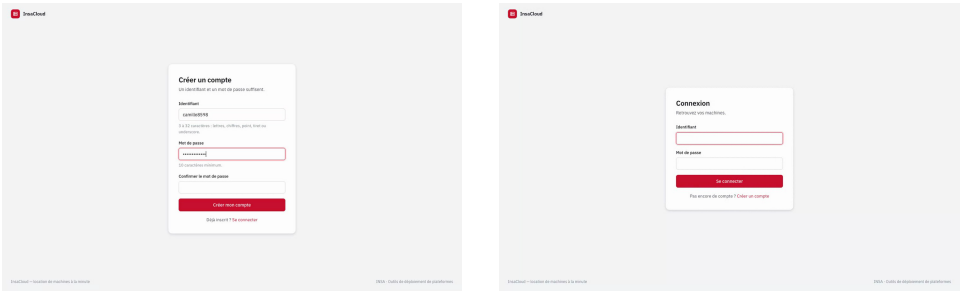


Figure 15.2 – Inscription et connexion. La politique de mot de passe est vérifiée côté serveur : un contrôle fait uniquement dans le navigateur se contourne en désactivant JavaScript.

15.4 Empreinte et chiffrement, à ne pas confondre

Notion — Deux besoins, deux mécanismes

Le mot de passe d'un *compte* est **haché** (`pbkdf2:sha256`) : l'opération est à sens unique, on ne peut pas le retrouver, seulement vérifier qu'un essai donne la même empreinte.
Le mot de passe *root d'une machine louée* est **chiffré** : il doit pouvoir être réaffiché, donc l'opération doit être réversible.
Hacher le second le rendrait inutilisable ; chiffrer le premier serait une faute.

Cinq échecs de connexion verrouillent temporairement le compte. Les tentatives sont enregistrées avec l'adresse IP, dans une table indexée.

15.5 Les en-têtes et la politique de contenu

En-tête	Effet
`Strict-Transport-Security`	le navigateur refuse ensuite le HTTP en clair
`X-Content-Type-Options`	interdit de deviner le type d'un fichier
`X-Frame-Options`	empêche l'inclusion du site dans un cadre
`Content-Security-Policy`	`default-src 'self'` : aucune ressource externe

À retenir

Une CSP stricte a un coût : plus de CDN, plus de script en ligne. Tailwind est donc précompilé et les polices sont hébergées localement. C'est plus de travail au départ, et cela supprime d'un coup toute une famille d'attaques par injection de script.

CHAPITRE 16

La haute disponibilité

16.1 Le scénario exact du cahier des charges

L'énoncé décrit un test précis : éteindre un worker et vérifier que l'instance repart ailleurs. Voici ce qui se passe, mesuré sur un cluster réel.

Instant	Événement
T + 0 s	`vagrant halt worker1` : l'agent cesse de battre
T + 90 s	le Faucheur déclare `worker1` `OFFLINE`
T + 90 s	la machine est recréée sur `worker2`, `migrations` passe à 1
T + 100 s	l'utilisateur voit un nouveau port et un nouveau mot de passe
plus tard	`worker1` redémarre, son agent se réinscrit en moins de 10 s

Listing 16.1 — `2_webapp/services.py` — la reprise

```
def recreer_ailleurs(instance: dict) -> Resultat:
    """
    Recree une instance sur un autre worker apres la panne du sien.

    Le conteneur est reconstruit a partir de la meme image, avec un mot
        ↪  de passe
    neuf - l'ancien vivait dans un conteneur devenu inaccessible. La
        ↪ location
    n'est pas touchee : l'utilisateur conserve la duree qu'il a payee.
    """
    ancien_noeud = instance["worker"]
    candidat = db.select_available_worker()
    if candidat is None or candidat["hostname"] == ancien_noeud:
        return Resultat.echec("Aucun worker de repli disponible.", code
            ↪ =503)
    ...
    db.move_instance(instance["id"], ports["container_id"],
        ↪ nouveau_noeud,
                     ports["port"], ports["term_port"], ports["gui_port
                         ↪ "])
    db.set_instance_password(instance["id"], encrypt_secret(
        ↪ mot_de_passe))
```

Noter ce que ce code *ne* fait pas : il ne touche pas à la table `rentals`. C'est tout l'intérêt de la séparation du chapitre 7.

16.2 Le compteur de migrations

La colonne `migrations` de la table `instances` s'incrémente à chaque reprise. Ce n'est pas décoratif : c'est la preuve observable qu'une reprise a eu lieu, et c'est ce que vérifie le test d'intégration.

> **À retenir**
>
> Quand une fonctionnalité est difficile à observer, ajoutez-lui un compteur. Sans `migrations`, il faudrait lire les journaux pour savoir si la reprise a fonctionné — et un test ne peut pas raisonnablement lire des journaux.

CHAPITRE 17

Sauvegarder le travail avant destruction

17.1 Le besoin

Détruire une machine détruit sa couche inscriptible. Un utilisateur qui a travaillé trente minutes perd tout. Ce n'est demandé nulle part dans le cahier des charges — c'est un ajout, proposé par un contributeur extérieur et intégré après correction.

17.2 Le principe : un flux, pas un fichier temporaire

```
docker exec <conteneur> tar czf - -C / root home
```

Le tiret à la place du nom de fichier demande à `tar` d'écrire sur la sortie standard. Python lit ce flux et l'écrit sur le disque du contrôleur. Pour un worker distant, le transport SSH du client Docker fait passer le flux : rien de plus n'est exposé sur le réseau, et rien n'est écrit dans le conteneur.

Listing 17.1 — `2_webapp/archives.py` — lire le flux sous contrainte

```
    # O_EXCL : on ne reecrit jamais un fichier existant. Mode 0600 des
        ↪ la
    # creation : l'archive peut contenir des cles ou des mots de passe.
    fd = os.open(destination, os.O_WRONLY | os.O_CREAT | os.O_EXCL, 0
        ↪ o600)
    with os.fdopen(fd, "wb") as sortie, subprocess.Popen(
            commande, stdout=subprocess.PIPE, stderr=subprocess.DEVNULL
                ↪ ) as processus:

        def _delai():
            delai_depasse.set()
            processus.kill()

        minuteur = threading.Timer(ARCHIVE_TIMEOUT, _delai)
        minuteur.start()
        try:
            while bloc := processus.stdout.read(TAILLE_BLOC):
                taille += len(bloc)
                if taille > ARCHIVE_MAX_BYTES:
                    processus.kill()
```

```
                    return {"status": STATUS_TOO_LARGE}
                sortie.write(bloc)
                empreinte.update(bloc)
        finally:
            minuteur.cancel()
```

Notion — Pourquoi un minuteur et pas un test dans la boucle

On pourrait vérifier le temps écoulé à chaque tour de boucle. Cela ne marcherait pas : `read()` *bloque* tant que Docker — ou la connexion SSH — n'envoie rien. Si le nœud se fige, la boucle ne tourne plus du tout et aucun test n'est évalué.
Un fil séparé qui tue le processus au bout du délai est la seule façon de garantir la sortie.

17.3 La règle qui prime sur tout

Listing 17.2 – `2_webapp/archives.py` — l'invariant du module

```
"""
Regle de conception : `archiver_conteneur` ne leve JAMAIS d'exception.
    ↪ Un
echec d'archivage (noeud injoignable, archive trop lourde, disque plein
    ↪ ) doit
etre signale a l'appelant, pas l'empecher de detruire la machine :
    ↪ sinon une
machine expiree resterait en vie indefiniment.
"""
```

À retenir

Une fonctionnalité secondaire ne doit jamais pouvoir bloquer une fonctionnalité essentielle. Ici, sauvegarder est un service rendu ; détruire à l'échéance est une obligation. Si l'un doit céder, ce n'est pas le second.

17.4 Retenter sans boucler

La première version ne retentait jamais un archivage raté : une simple coupure SSH condamnait les fichiers. Mais retenter à chaque cycle du Faucheur serait pire : sur un nœud durablement muet, chaque passage attendrait le délai maximal de 45 secondes, toutes les 10 secondes.

Listing 17.3 – `2_webapp/database.py` — le compromis

```
def archivage_deja_fait(instance_id: int, delai_avant_reprise: int =
    ↪ 600) -> bool:
    """
    Une archive reussie n'est jamais refaite : la destruction peut etre
    retentee pendant plusieurs cycles du Faucheur, le travail n'est
        ↪ copie
    qu'une fois.

    Un echec, en revanche, vient parfois d'un incident passager - un
        ↪ noeud
    brievement injoignable. On retente alors, mais pas a chaque cycle.
    """
```

Notion — Le téléchargement, et ses trois protections

Une archive contient les fichiers privés d'un utilisateur. La route de téléchargement les protège de trois façons :

1. le propriétaire est vérifié *dans la requête SQL* : l'archive d'un autre compte répond 404, exactement comme une archive inexistante — cela ne révèle rien sur les comptes voisins ;
2. `send_from_directory` joint le chemin au dossier des archives et refuse toute sortie (`../`), même si la base était altérée ;
3. la réponse est un téléchargement (`attachment`), jamais affichée dans la page.

Quatrième partie

Le déploiement

CHAPITRE 18

Monter le parc avec Vagrant

18.1 Quatre machines décrites en un fichier

Listing 18.1 — `Vagrantfile` — le parc, paramétrable

```
# Trois workers, comme le demande le cahier des charges. Sur un poste
    ↪ modeste,
# WORKERS=2 (ou 1) reduit le parc sans toucher au fichier :
#   WORKERS=2 vagrant up
NB_WORKERS = (ENV["WORKERS"] || "3").to_i.clamp(1, 9)

MACHINES = {
  "controller" => { "ip" => "192.168.56.10", "memory" => 2048, "cpus"
      ↪ => 2 },
}
(1..NB_WORKERS).each do |n|
  MACHINES["worker#{n}"] = { "ip" => "192.168.56.#{10 + n}",
                             "memory" => 3072, "cpus" => 2 }
end
```

> **À retenir**
>
> Rendre le nombre de workers paramétrable coûte trois lignes et change tout pour qui lit le projet : on peut le faire tourner sur un portable avec un seul worker, ou monter à neuf pour une démonstration. Un fichier qui n'offre qu'une seule taille finit toujours par être modifié à la main — et donc désynchronisé.

18.2 Plusieurs providers, et pourquoi

```
node.vm.provider "vmware_desktop" do |v| ... end
node.vm.provider "parallels"      do |p| ... end
node.vm.provider "qemu"           do |q, override| ... end
node.vm.provider "virtualbox"     do |v| ... end
```

VirtualBox, le provider par défaut, ne fonctionne pas sur les Mac à puce Apple. Déclarer plusieurs providers permet au même fichier de marcher sur une machine Intel, un Mac ARM et un poste Windows.

Piège — QEMU ne gère pas les réseaux privés

Chaque provider a ses limites. `vagrant-qemu` ne sait pas créer de réseau privé : l'accès SSH est redirigé sur `localhost`. Le `Vagrantfile` le détecte et ajuste la configuration. Sans cela, le déploiement réussirait et rien ne serait joignable.

18.3 Les commandes du quotidien

```
vagrant up                  # cree et demarre tout le parc
vagrant up worker2          # une seule machine
vagrant status              # qui tourne
vagrant ssh controller      # entrer dans une machine
vagrant halt worker1        # eteindre (le test de panne du cahier des
    ↪ charges)
vagrant reload worker1      # redemarrer
vagrant destroy -f          # tout effacer, pour repartir de zero
```

CHAPITRE 19

Configurer avec Ansible

19.1 La partition : quatre plays, neuf rôles

Listing 19.1 – `3_ansible/site.yml` — la structure

```
- name: 1. Durcissement de tous les noeuds (UFW)
  hosts: insacloud
  become: true
  roles:
    - role: security_hardening

- name: 2. Controleur - compte de service et cle SSH de pilotage
  hosts: controllers
  roles:
    - role: controller_key

- name: 3. Workers - Docker, certificat, acces du controleur, images
  hosts: workers
  roles:
    - role: docker
    - role: tls_cert
    - role: worker
    - role: docker_images
    - role: worker_agent

- name: 4. Controleur - client Docker, certificat, nginx, application
  hosts: controllers
  roles:
    - role: docker
    - role: tls_cert
    - role: reverse_proxy
    - role: webapp
```

Notion — Pourquoi cet ordre

L'ordre n'est pas arbitraire. Le contrôleur doit avoir sa clé SSH *avant* que les workers n'autorisent cette clé ; les workers doivent avoir Docker *avant* qu'on y construise les images ; nginx doit être configuré *avant* que l'application démarre derrière lui.
Un playbook est une suite de dépendances. Les écrire explicitement évite les « ça marche une fois sur deux ».

Rôle	Ce qu'il fait
`security_hardening`	pare-feu UFW, ports ouverts selon le groupe
`controller_key`	compte de service et clé SSH de pilotage
`docker`	Docker Engine sur les workers, client seul sur le contrôleur
`tls_cert`	certificat auto-signé propre à chaque nœud
`worker`	accès SSH restreint du contrôleur, appartenance au groupe docker
`docker_images`	construction des images des machines louées
`worker_agent`	l'agent d'inscription et de battement de cœur
`reverse_proxy`	nginx HTTPS devant Gunicorn
`webapp`	Flask, Gunicorn, Faucheur, watchdog, secrets

19.2 Un piège réel : la déduplication des rôles

Ansible n'exécute un rôle qu'une fois par play. Sauf si on l'appelle avec des *paramètres différents* : il le considère alors comme deux rôles distincts et l'exécute deux fois.

Le rôle `docker` était appelé directement par le play du contrôleur, et tiré en dépendance par `docker_images`. Avec un paramètre différent à chaque appel, il s'exécutait deux fois — installant puis désinstallant le démon.

Listing 19.2 – La correction : une variable de play, pas un paramètre de rôle

```
- name: 4. Controleur
  hosts: controllers
  vars:
    single_host: "{{ groups['workers'] | default([]) | length == 0 }}"
    # Defini ici (et non en parametre de role) : un role appele avec des
    # parametres differents n'est pas dedoublonne par Ansible, et `docker` est
    # aussi tire comme dependance par `docker_images`. Une variable de play
    # s'applique aux deux appels, qui sont donc fusionnes en une execution.
    docker_cli_only: "{{ not single_host }}"
  roles:
    - role: docker
    - role: docker_images
      when: single_host
```

À retenir

Quand un comportement d'outil vous surprend, vérifiez-le par l'expérience plutôt que par la documentation. Nous avons prouvé celui-ci en instrumentant le rôle pour qu'il journalise chaque exécution.

19.3 Les secrets : Ansible Vault

```
# Creer ou editer un fichier chiffre
ansible-vault create group_vars/insacloud/vault.yml
ansible-vault edit   group_vars/insacloud/vault.yml

# Jouer le playbook en fournissant la phrase de passe
ansible-playbook site.yml --ask-vault-pass
```

Le fichier est chiffré en AES-256 et peut être versionné sans danger. Il contient la clé de session Flask, la clé du coffre Fernet, le mot de passe PostgreSQL et le jeton des agents.

> **Piège — Un secret déposé dans une unité systemd**
>
> Mettre `Environment=INSACLOUD_VAULT_KEY=...` dans un fichier d'unité le rend lisible par n'importe qui : les unités sont en lecture pour tous, et `systemctl show` les affiche.
> La bonne façon : un fichier séparé en `0600`, appartenant au compte de service, référencé par `EnvironmentFile=`. Le secret n'est alors ni dans l'unité, ni dans les journaux.

19.4 Prouver l'idempotence

```
ansible-playbook site.yml --ask-vault-pass        # premier passage
ansible-playbook site.yml --ask-vault-pass        # second passage
```

Le second passage doit afficher `changed=0` sur chaque nœud. C'est mesuré sur les quatre machines après chaque modification du projet.

```
PLAY RECAP *********************************************************************
controller : ok=71  changed=0  unreachable=0  failed=0  skipped=9
worker1    : ok=43  changed=0  unreachable=0  failed=0  skipped=4
worker2    : ok=43  changed=0  unreachable=0  failed=0  skipped=4
worker3    : ok=43  changed=0  unreachable=0  failed=0  skipped=4
```

> **Piège — `command` et `shell` ne sont jamais idempotents**
>
> Les modules `ansible.builtin.command` et `shell` exécutent ce qu'on leur donne, sans rien vérifier : ils sont donc *toujours* signalés `changed`. Si vous devez les employer, ajoutez `creates:`, `removes:` ou `changed_when:` pour leur apprendre à se taire quand il n'y a rien à faire.

19.5 L'inventaire dynamique

L'inventaire statique liste les machines à la main. Un inventaire *dynamique* les découvre.

```
./inventaire_vagrant.py --list      # interroge Vagrant, produit du
    ↪ JSON
ansible-playbook -i inventaire_vagrant.py site.yml
```

Le script interroge `vagrant status` et construit les groupes. Changer le nombre de VM ne demande plus de toucher à l'inventaire, et le déploiement devient indépendant du provider de virtualisation.

CHAPITRE 20

Servir l'application

20.1 nginx devant Gunicorn

Listing 20.1 – Le principe de la configuration nginx

```
server {
    listen 443 ssl http2;
    ssl_protocols TLSv1.3;

    add_header Strict-Transport-Security "max-age=63072000" always;
    add_header X-Content-Type-Options nosniff always;

    location /static/ {
        alias /opt/insacloud/static/;    # servi par nginx, pas par
            ↪ Python
        expires 7d;
    }

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Notion — `X-Forwarded-For` et `ProxyFix`

Derrière un proxy, toutes les requêtes semblent venir de `127.0.0.1`. L'adresse réelle du client est placée par nginx dans l'en-tête `X-Forwarded-For`.
Côté Flask, le *middleware* `ProxyFix` lit ces en-têtes et rétablit la bonne adresse. Sans lui, notre verrouillage anti-force-brute compterait toutes les tentatives comme venant de la même IP — et bloquerait tout le monde au cinquième échec de n'importe qui.

20.2 systemd : faire tourner les services

Listing 20.2 – Le principe d'une unité systemd

```
[Unit]
```

```
Description=InsaCloud - application web
After=network-online.target

[Service]
User=insacloud
WorkingDirectory=/opt/insacloud
EnvironmentFile=/etc/insacloud/insacloud.env     # secrets, en 0600
ExecStart=/opt/insacloud/.venv/bin/gunicorn --preload -w 4 -b
    ↪ 127.0.0.1:5000 app:app
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
```

Directive	Effet
`User=`	le service ne tourne pas en root
`EnvironmentFile=`	les secrets viennent d'un fichier protégé, pas de l'unité
`Restart=always`	systemd relance le service s'il meurt
`After=network-online`	ne pas démarrer avant que le réseau soit prêt
`--preload`	Gunicorn importe l'application une fois, puis duplique

20.3 Le watchdog

Un *timer* systemd vérifie chaque minute que nginx répond, que le site renvoie bien du HTTPS, et que le Faucheur tourne. Il redémarre ce qui est tombé et journalise les workers injoignables.

> **Notion — Timer plutôt que cron**
>
> Un timer systemd s'intègre aux journaux (`journalctl -u ...`), connaît l'état du service qu'il surveille, et ne démarre pas avant que ses dépendances soient prêtes. Cron ne sait rien de tout cela.

CHAPITRE 21

La pile `docker compose`

21.1 Pourquoi une seconde façon de déployer

Monter quatre VM demande du temps, de la mémoire et un hyperviseur. Pour développer, démontrer, ou faire tourner les tests d'intégration en intégration continue, on veut une pile en une commande.

```
cp .env.example .env          # puis renseigner les secrets
docker compose up -d
```

21.2 Un bloc partagé par deux services

Listing 21.1 – `docker-compose.yml` — l'ancre YAML

```
x-application: &application
  build:
    context: ./2_webapp
  image: insacloud_web:latest
  restart: unless-stopped
  environment:
    INSACLOUD_ENV: production
    INSACLOUD_CLUSTER: ${INSACLOUD_CLUSTER:-compose}
    DATABASE_URL: "postgresql+psycopg://...:${POSTGRES_PASSWORD:?
        ↪ manquant}@db:5432/..."
    INSACLOUD_SECRET_KEY: ${INSACLOUD_SECRET_KEY:?definissez
        ↪ INSACLOUD_SECRET_KEY}
  depends_on:
    db:
      condition: service_healthy

services:
  web:
    <<: *application              # reprend tout le bloc ci-dessus
    ports: ["127.0.0.1:8088:5000"]
  faucheur:
    <<: *application
    command: ["python", "faucheur.py"]
```

> **Notion — Les ancres YAML**
>
> `&application` donne un nom à un bloc ; `<<: *application` le recopie. L'application et le Faucheur partagent ainsi exactement la même confi-

guration — il devient impossible qu'ils divergent sur une variable.

Notion — `${VAR:?message}`

Cette syntaxe refuse de démarrer si la variable n'est pas définie, en affichant le message. La pile ne peut donc pas tourner avec un mot de passe vide ou une clé par défaut : l'erreur arrive au démarrage, pas trois semaines plus tard.

21.3 Le volume partagé, et pourquoi il ne s'invente pas

```
  web:
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - archives_travail:/var/lib/insacloud/archives
  faucheur:
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - archives_travail:/var/lib/insacloud/archives

volumes:
  donnees_postgres:
  archives_travail:
```

Piège — Deux services, un seul dossier

Le Faucheur *écrit* l'archive ; l'application la *sert* au téléchargement. Si chacun avait son propre volume, l'utilisateur verrait dans son tableau de bord une archive que le serveur ne trouverait pas.
Le même volume nommé doit donc être monté dans les deux services. C'est le genre d'erreur qui ne se voit pas à la lecture du fichier, et qui saute aux yeux au premier essai réel.

Piège — La socket Docket donne les pleins pouvoirs

Monter `/var/run/docker.sock` dans un conteneur lui donne un pouvoir équivalent à root sur l'hôte : il peut créer un conteneur privilégié qui monte le disque entier. C'est acceptable sur un poste de développement, jamais en production — où le contrôleur pilote des workers distants par SSH et n'a plus besoin de cette ligne.

21.4 Les profils

```
docker compose up -d                          # la plateforme seule
docker compose --profile build build          # + construire les six
    ↪ images
docker compose --profile demo up -d           # + une machine de
    ↪ demonstration
```

Un service marqué d'un profil ne démarre que si on le demande. Cela permet de décrire dans un seul fichier la pile minimale et tous ses compléments.

Cinquième partie

La vérification

CHAPITRE 22

Les tests

22.1 Tester Docker sans Docker

Le problème est concret : les tests doivent tourner sur un agent d'intégration qui n'a pas forcément Docker, et ils doivent être rapides. Lancer de vrais conteneurs prendrait des minutes et échouerait au moindre réseau capricieux.

La solution : remplacer `run_docker` par une fonction espion qui note ce qu'on lui demande, sans rien exécuter.

Listing 22.1 – Le principe de l'espion

```
def test_la_commande_imposee_est_bien_celle_qui_part(monkeypatch):
    appels = []
    monkeypatch.setattr(docker_ops, "run_docker",
                        lambda *a, **k: appels.append(a) or "
                            ↪ abc123def456")

    docker_ops.docker_run_container(port=8001, name="insacloud_test",
        ↪ ...)

    args = appels[0]
    assert "run" in args and "-d" in args
    assert "--restart=always" in args
    assert "-p" in args and "8001:22" in args
```

Notion — *monkeypatch* et double de test

`monkeypatch` est une fonctionnalité de pytest qui remplace un attribut le temps d'un test, puis le restaure. On parle de *double de test* : un objet qui prend la place du vrai pour la durée de l'essai.
Ici, cela permet de vérifier que la commande imposée par l'énoncé — `docker run -d --restart=always -p <port>:22` — est bien celle qui part, sur une machine qui n'a pas Docker.

22.2 Partir d'une base vierge à chaque test

Listing 22.2 – `2_webapp/tests/conftest.py`

```
@pytest.fixture(autouse=True)
def base_vierge():
```

```
    """
    Chaque test part d'une base vide : aucun test n'herite de l'etat d'
        ↪ un autre.

    Le moteur SQLAlchemy est ferme avant d'effacer le fichier : sans
        ↪ cela, les
    connexions du pool continueraient de pointer sur l'ancien fichier
        ↪ efface.
    """
    db.engine.dispose()
    try:
        os.remove(db.DB_PATH)
    except FileNotFoundError:
        pass
    db.init_db()
    yield
```

Piège — Des tests qui dépendent de leur ordre

Si un test laisse des données derrière lui, le suivant peut réussir pour la mauvaise raison — ou échouer quand on le lance seul. Le symptôme : « ça passe en entier mais pas tout seul », ou l'inverse.
`autouse=True` applique la remise à zéro à *tous* les tests, sans qu'on ait à y penser.

22.3 Préparer l'environnement avant l'import

Listing 22.3 – `2_webapp/tests/conftest.py` — l'ordre compte

```
# --- Environnement de test, fixe AVANT l'import de l'application
    ↪ -------------
_TMP = tempfile.mkdtemp(prefix="insacloud-tests-")
os.environ["INSACLOUD_DB"] = os.path.join(_TMP, "test.db")
os.environ["INSACLOUD_SECRET_KEY"] = "cle-de-session-de-test-uniquement
    ↪ "
os.environ["INSACLOUD_VAULT_KEY"] = Fernet.generate_key().decode()
os.environ["INSACLOUD_WORKERS"] = ""          # mode mono-hote

import pytest                                 # noqa: E402
import app as insacloud                       # noqa: E402
```

À retenir

L'application lit sa configuration *au moment de l'import*. Si l'on importait `app` avant de poser ces variables, les tests écriraient dans la vraie base du poste de développement et utiliseraient les vraies clés. L'ordre des lignes de ce fichier est donc une mesure de sécurité, pas un détail

de style.

22.4 Ce que couvrent les 168 tests

Fichier	Tests	Ce qu'il vérifie
`test_securite.py`	32	CSRF, anti-force-brute, cloisonnement entre comptes, en-têtes, coffre
`test_workers.py`	37	inscription, battement de cœur, états, ordonnanceur, reprise, libération du nom
`test_docker.py`	24	la commande construite, le durcissement, les labels
`test_faucheur.py`	17	expiration, réconciliation par nœud, nœud injoignable
`test_contraintes.py`	7	index unique partiel, unicité, clés étrangères, idempotence
`test_archives*.py`	51	archivage, purge, téléchargement, reprise après échec
Total	**168**	en 20 secondes, sans Docker ni réseau

22.5 Le test de cloisonnement, le plus important

Listing 22.4 – Vérifier qu'un compte ne voit pas les machines d'un autre

```
def test_un_utilisateur_ne_voit_pas_la_machine_d_un_autre(client):
    # Alice loue une machine
    inscrire(client, "alice"); loue = louer(client)
    client.post("/logout")

    # Bob se connecte et tente d'y acceder par son identifiant
    inscrire(client, "bob")
    reponse = client.get(f"/instances/{loue['id']}")

    # 404, pas 403 : repondre « interdit » confirmerait que la machine
        ↪ existe
    assert reponse.status_code == 404
```

Notion — 404 plutôt que 403

Répondre « accès interdit » révèle que la ressource existe. En énumérant les identifiants, un attaquant apprendrait combien de machines tournent et lesquelles. Répondre « introuvable » ne distingue pas « n'existe pas » de « ne vous appartient pas ».
C'est pourquoi le filtre sur le propriétaire est placé *dans la requête SQL* plutôt que dans un test après coup : il devient impossible d'oublier.

22.6 Les tests d'intégration : la vraie pile

Les tests unitaires ne lancent rien. Un second niveau monte la pile complète et la parcourt :

```
./tests/integration.sh http://127.0.0.1:8088
```

Scénario	Ce qu'il prouve
1. Infrastructure	les nœuds sont là et se parlent
2. Configuration	Ansible a fait son travail
3. Application	inscription, connexion, tableau de bord
4. Location	louer une machine et s'y connecter en SSH
5. Expiration	une location échue est réellement détruite
6. Panne	un nœud tombe, la machine repart ailleurs

Piège — Le port est publié avant que le service écoute

Docker publie le port dès la création du conteneur, mais `sshd` met une seconde ou deux à démarrer. Un test qui se connecte immédiatement échoue de façon intermittente.
La correction n'est pas d'attendre « assez longtemps » — une durée fixe est toujours trop courte un jour et trop longue tous les autres — mais de réessayer jusqu'à ce que le service réponde, avec une limite.

CHAPITRE 23

La chaîne d'intégration

23.1 Onze travaux

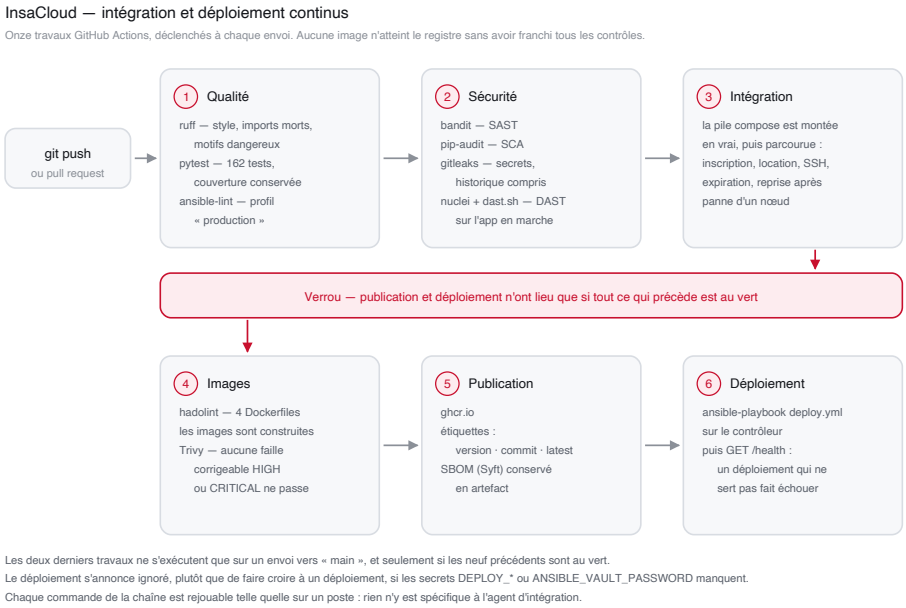


Figure 23.1 – Les onze travaux et le verrou qui protège le registre.

Travail	Outil	Ce qu'il attrape
Lint Python	`ruff`	imports morts, noms inconnus, motifs douteux
Tests	`pytest`	168 cas, couverture conservée en artefact
SAST	`bandit`	injection, secrets en dur, appels système risqués
SCA	`pip-audit`	CVE des dépendances épinglées
Secrets	`gitleaks`	clés commises, y compris dans l'historique
DAST	`dast.sh` + `nuclei`	l'application en fonctionnement
Lint Ansible	`ansible-lint`	rôles, profil `production`
Images	`hadolint` + `Trivy`	4 Dockerfiles, CVE des images
Intégration	`docker compose`	la pile démarre et une machine se loue
Publication	`Syft` + `ghcr.io`	SBOM conservé, images étiquetées
Déploiement	`ansible-playbook`	met à jour l'hôte puis interroge `/health`

23.2 Le verrou

Les deux derniers travaux ne s'exécutent que sur un envoi vers `main`, et seulement si les neuf précédents sont au vert :

```
publier-images:
  needs: [lint-python, tests-python, sast-python, sca-dependances,
      ↪ secrets,
          dast, lint-ansible, images-docker, integration]
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
```

> **À retenir**
>
> Sans `needs:`, GitHub Actions lance tous les travaux en parallèle — et publierait l'image pendant que les tests échouent. La dépendance explicite est ce qui transforme une liste de contrôles en une véritable barrière.

23.3 Trois incidents instructifs

Nos images refusées par notre propre scan. Le travail `images-docker` refuse toute faille corrigeable de niveau HIGH ou CRITICAL. Notre image utilisait `docker:28-cli`, qui traînait 163 failles corrigeables liées à sa version de Go, plus des CVE OpenSSL héritées de la base Debian.

La correction a tenu en deux gestes : passer à `docker:29-cli` (2 failles au lieu de 163) et ajouter un `apt-get upgrade` pour OpenSSL. Le verrou utilise par ailleurs `--ignore-unfixed` : refuser une faille sans correctif disponible ne protégerait de rien et bloquerait tout.

Une dépendance qui n'existe pas pour la cible. Le projet épinglait SQLAlchemy 2.1.1, disponible sur notre poste. La cible de déploiement est Ubuntu 22.04, qui fournit Python 3.10 — pour lequel cette version n'a jamais été publiée. Les tests passaient, le déploiement échouait.

Deux guillemets déséquilibrées. Une erreur de syntaxe shell dans un bloc `run:` ne se voit qu'au moment où l'étape s'exécute, parfois après dix minutes de chaîne. Trois exécutions ont échoué ainsi. Nous avons ajouté un contrôle qui extrait chaque bloc `run:`, neutralise les expressions `${{ }}` et passe le tout à `bash -n` :

Listing 23.1 — `.github/workflows/ci.yml` — la chaîne vérifie sa propre syntaxe

```
      - name: Verifier la syntaxe shell de la chaine elle-meme
        run: |
          pip install --quiet pyyaml
```

```
          python3 .github/verifier-workflow.py
```

À retenir

Chaque fois qu'un défaut vous a coûté du temps, demandez-vous quel contrôle automatique l'aurait attrapé — puis ajoutez-le. Une chaîne d'intégration n'est pas écrite d'un coup : elle se construit par sédimentation, un incident après l'autre.

23.4 Empêcher la documentation de vieillir

Les schémas du dépôt sont engendrés par des scripts Python ; la chaîne les régénère et refuse la moindre différence :

```
      - name: Verifier que les schemas du README sont a jour
        run: |
          python3 docs/schemas/tout.py
          git diff --exit-code -- docs/*.svg
```

Piège — Du texte à jour, des images périmées

Ce garde-fou vient d'une erreur réelle. Le README avait été mis à jour — texte, chiffres, descriptions d'images — mais les schémas SVG, eux, montraient encore deux workers alors que le projet en comptait trois.
La moitié du contenu d'un README est visuelle. La traiter comme du texte, c'est garantir qu'elle finira fausse.

CHAPITRE 24

Ce qu'on a trouvé en faisant tourner

Les défauts qui suivent n'ont pas été trouvés en relisant le code, mais en le faisant fonctionner : en déployant sur un cluster, en lançant la pile, en lisant les journaux.

Défaut	Garde-fou ajouté
Le Faucheur détruisait sa propre plateforme : il reconnaissait les machines à leur préfixe de nom	Identité par étiquette Docker, plus jamais par nom ; un test le vérifie
Deux déploiements sur un même démon se supprimaient mutuellement leurs conteneurs	Un second label délimite le déploiement
Le mot de passe de la base journalisé en clair au démarrage du Faucheur	Toute URL passe par une fonction de masquage
Une dépendance épinglée inexistante pour le Python de la cible	La chaîne installe exactement `requirements.txt` sur la cible réelle
Des distributions proposées sans image construite	Le catalogue interroge Docker avant d'afficher un choix
Course au démarrage entre les quatre processus Gunicorn	Création tolérante à la concurrence, et `--preload`
Images refusées par notre propre scan Trivy	Base plus récente, `apt-get upgrade`, `--ignore-unfixed`
Deux guillemets déséquilibrées dans la chaîne elle-même	Un contrôle de syntaxe shell de tous les blocs `run:`
Le mot de passe root lisible dans le cookie de session	Seul l'identifiant transite ; le secret est relu chiffré en base
Les schémas du README figés sur une ancienne architecture	La chaîne régénère les SVG et refuse toute différence

À retenir

Dix défauts, dix garde-fous. C'est la différence entre « on a corrigé » et « cela ne peut plus revenir ». Corriger sans ajouter de contrôle, c'est accepter de recommencer.

24.1 Ce qui a été vérifié sur un cluster réel

— **Déploiement complet** : `failed=0` sur les quatre nœuds.
— **Idempotence** : second passage à `changed=0` sur les quatre.
— **Inscription spontanée** : les trois agents se sont enregistrés seuls, aucune adresse de worker n'étant écrite côté contrôleur.
— **Parcours utilisateur** : inscription, location, accès ; le conteneur est bien créé sur le nœud le moins chargé.
— **Durcissement effectif** : `docker inspect` confirme `restart=always`, les limites mémoire, `pids-limit`, `cap-drop ALL` et `no-new-privileges`.
— **Conservation de l'état** : après redémarrage du démon Docker *puis* du nœud entier, la machine et le travail de l'utilisateur sont intacts.
— **Faucheur** : une location d'une minute détruite 49 secondes après son expiration.
— **Reprise sur panne** : le scénario de l'énoncé joué tel quel.
— **Analyse dynamique** : 18 contrôles sur l'application en fonctionnement, tous réussis.

Sixième partie

Annexes

CHAPITRE 25

Refaire le projet depuis zéro

Voici l'ordre dans lequel aborder un projet de ce type. Chaque étape produit quelque chose qui marche : c'est ce qui permet de ne jamais se retrouver avec mille lignes dont rien ne fonctionne.

1. **Un conteneur SSH à la main.** Écrivez un Dockerfile qui installe `openssh-server`, lancez-le avec `-p 2222:22`, connectez-vous. Objectif : comprendre la publication de port.
2. **Plusieurs services dedans.** Ajoutez `supervisord` et un terminal web. Objectif : comprendre PID 1.
3. **Une page Flask qui lance ce conteneur.** Un bouton, un `subprocess`, un message. Pas de base, pas de comptes.
4. **Une base.** Enregistrez ce que vous créez. Séparez dès maintenant la machine de la location : ce choix est difficile à rattraper après coup.
5. **Des comptes.** Inscription, connexion, et surtout le cloisonnement — un utilisateur ne voit que ses machines.
6. **L'expiration.** Un script séparé qui boucle et détruit. Testez-le avec des locations d'une minute.
7. **Les tests.** Maintenant, pas plus tard : vous avez assez de code pour que les régressions commencent.
8. **Un second nœud.** C'est là que tout change : SSH, ordonnanceur, agent. Faites-le avant d'avoir trop de code supposant un hôte unique.
9. **Ansible.** Automatisez ce que vous faites à la main depuis le début. Visez `changed=0` au second passage dès le premier jour.
10. **La chaîne d'intégration.** Commencez par lint et tests, ajoutez le reste progressivement.
11. **La haute disponibilité.** Éteignez un nœud et regardez ce qui se passe. Corrigez. Recommencez.

> **À retenir**
>
> L'ordre compte plus qu'on ne croit. Ajouter le multi-nœuds à la fin oblige à réécrire tout le code qui appelait `docker` sans se demander « sur quelle machine ? ». Ajouter les tests à la fin revient à ne pas les ajouter.

CHAPITRE 26

Commandes utiles

26.1 Le projet

```
# --- Pile complete en local
    ↪ ------------------------------------------------
cp .env.example .env                        # puis renseigner les secrets
docker compose up -d
docker compose --profile build build        # construire les six images
docker compose logs -f web
docker compose down -v                      # tout effacer, volumes
    ↪ compris

# --- Developpement
    ↪ ------------------------------------------------------------
cd 2_webapp
python3 -m venv .venv && .venv/bin/pip install -r requirements-dev.txt
.venv/bin/python app.py                     # serveur de developpement

# --- Controles, les memes que la chaine
    ↪ --------------------------------------
pytest                                      # 168 tests
ruff check .
bandit -c pyproject.toml -r .
pip-audit -r requirements.txt
alembic check
./tests/dast.sh http://127.0.0.1:5000
./tests/integration.sh http://127.0.0.1:8088

# --- Cluster
    ↪ ------------------------------------------------------------------
    ↪
vagrant up
cd 3_ansible && ansible-playbook site.yml --ask-vault-pass
ansible-playbook site.yml --ask-vault-pass    # doit donner changed=0
vagrant halt worker1                        # le test de panne
```

26.2 Diagnostiquer

```
# Que fait reellement un conteneur ?
docker inspect <conteneur> | jq '.[0].HostConfig | {Memory, NanoCpus,
    ↪ CapDrop, PidsLimit}'

# Pourquoi ne demarre-t-il pas ?
```

```
docker logs <conteneur>

# Quelles machines la plateforme reconnait-elle comme siennes ?
docker ps -a --filter "label=insacloud.role=rented-machine"

# Les services sur le controleur
systemctl status insacloud insacloud-faucheur nginx
journalctl -u insacloud-faucheur -f

# L'etat du parc, vu de l'exterieur
curl -s http://127.0.0.1:8088/health | jq
curl -s -H 'Accept: application/json' http://127.0.0.1:8088/workers |
    ↪ jq
```

CHAPITRE 27

Erreurs fréquentes

Symptôme	Cause probable
`exec format error` ou `no such file or directory` sur un script qui existe pourtant	Le script a des fins de ligne Windows (CRLF). Le `\r` fait partie du nom de l'interpréteur dans le shebang. Corriger avec un `.gitattributes` : `* text=auto eol=lf`
`port is already allocated`	Un conteneur précédent n'a pas été nettoyé, ou deux requêtes ont choisi le même port. L'index unique partiel règle le second cas
`table users already exists` au démarrage	Plusieurs processus Gunicorn créent le schéma en même temps. Utiliser `--preload` et tolérer l'erreur
Le conteneur démarre puis s'arrête aussitôt	Son processus principal s'est terminé. Un conteneur vit tant que son PID 1 vit
`permission denied` sur `/var/run/docker.sock`	Le compte n'est pas dans le groupe `docker`. En conteneur, utiliser `group_add` avec le GID de l'hôte
Ansible affiche `changed` à chaque passage	Un module `command` ou `shell` sans `creates:` ni `changed_when:`
Le site répond en HTTP mais pas en HTTPS	Certificat absent ou illisible par nginx, ou port 443 fermé par UFW
Tout le monde est bloqué par l'anti-force-brute	`ProxyFix` n'est pas actif : toutes les requêtes semblent venir du proxy
Les tests passent ensemble mais échouent seuls (ou l'inverse)	Un test laisse des données derrière lui. Remettre la base à zéro avec une fixture `autouse`
La connexion SSH échoue juste après la location	Docker publie le port avant que `sshd` écoute. Réessayer avec une limite, plutôt qu'attendre une durée fixe

CHAPITRE 28

Conformité au cahier des charges

#	Fonctionnalité	Réalisation
1	Préparation et environnement	dépôt Git, environnement virtuel, dépendances épinglées
2	API Web Flask	modules séparés, comptes, sessions, tableau de bord
3	Base de données	7 tables en SQLAlchemy, migrations Alembic, index métier
4	Conteneurisation Docker	4 Dockerfiles, 6 images, durcissement complet
5	Infrastructure Vagrant	4 VM, réseau privé, 4 providers
6	Automatisation Ansible	9 rôles, 4 playbooks, Vault, idempotence mesurée
7	Worker Agent	inscription spontanée, battement de cœur, 3 états
8	Ordonnanceur	nœud joignable le moins chargé, départage déterministe
9	Service de location	durée 1–120 min, prolongation, arrêt
10	Déploiement des instances	la commande imposée, par `subprocess`
11	Accès SSH	par mot de passe ou par clé, plus terminal web et bureau
12	Expiration	le Faucheur, toutes les 10 s, avec réconciliation
13	Haute disponibilité	3 niveaux, scénario de l'énoncé vérifié
14	Sécurité DevSecOps	SAST, SCA, secrets, DAST, scan d'image, SBOM
15	CI/CD	11 travaux, publication conditionnelle, déploiement vérifié
16	Tests d'intégration	168 tests unitaires + 6 scénarios de bout en bout
17	Documentation	README, guide d'API, schéma de base, ce guide

Au-delà de l'énoncé

— Architecture distribuée à trois workers, alors qu'un seul hôte suffisait.

— Terminal web et bureau graphique complet dans le navigateur, en plus du SSH demandé.

— Coffre à secrets : mot de passe root chiffré, réaffichable après re-

authentification.
— Watchdog systemd d'auto-réparation du service.
— Inventaire dynamique interrogeant Vagrant.
— Sauvegarde du travail avant destruction, téléchargeable sept jours.
— Schémas engendrés depuis leur description, et contrôlés par la chaîne.

CHAPITRE 29

Glossaire

Agent petit programme tournant sur chaque worker, qui l'inscrit auprès du contrôleur et signale régulièrement sa présence.

Alembic outil de migration de schéma pour SQLAlchemy.

Ansible outil de configuration qui applique un état décrit en YAML, par SSH, de façon idempotente.

Capacité (*capability*) fraction des pouvoirs de root, attribuable indépendamment des autres.

cgroup mécanisme du noyau Linux limitant les ressources qu'un groupe de processus peut consommer.

CSP *Content Security Policy* : en-tête qui restreint les sources de scripts, styles et images qu'une page peut charger.

CSRF attaque où un site tiers fait exécuter une action au nom d'un utilisateur connecté ailleurs.

CVE identifiant mondial d'une faille de sécurité connue.

DAST analyse de sécurité de l'application *en fonctionnement*.

Faucheur le démon du projet qui détruit les machines expirées, réconcilie Docker et la base, et surveille le parc.

Fernet recette de chiffrement symétrique authentifié de la bibliothèque `cryptography`.

Gunicorn serveur WSGI multi-processus qui exécute l'application Flask en production.

Hachage transformation à sens unique d'un mot de passe, non réversible.

Idempotence propriété d'une opération dont la deuxième application ne change rien de plus que la première.

Image modèle figé en lecture seule, à partir duquel on crée des conteneurs.

Namespace mécanisme du noyau Linux qui cloisonne ce qu'un processus peut voir.

ORM bibliothèque faisant correspondre classes Python et tables SQL.

Proxy inverse serveur placé devant l'application, qui reçoit les requêtes à sa place.

SAST analyse de sécurité du code source, sans l'exécuter.

SBOM inventaire exhaustif des composants d'un logiciel livré.

SCA confrontation des dépendances aux bases de vulnérabilités connues.

supervisord gestionnaire de processus utilisé comme PID 1 dans les machines louées.

Vagrant outil de création de machines virtuelles décrites dans un fichier.

Vault fichier de secrets chiffré par Ansible, versionnable sans danger.

WSGI contrat d'interface entre un serveur Python et une application Python.

CHAPITRE 30

Pour aller plus loin

30.1 Ce que ce projet ne fait pas

Pas de stockage partagé la reprise après panne ne rend pas les fichiers. Un volume répliqué (NFS, Ceph, Longhorn) le permettrait, au prix d'une complexité bien supérieure.

Le contrôleur reste un point unique s'il tombe, le tableau de bord disparaît et les locations cessent d'expirer. Les machines louées, elles, continuent de tourner. Une vraie haute disponibilité demanderait plusieurs contrôleurs et une base répliquée.

Certificat auto-signé remplaçable par `mkcert` ou une autorité interne ; Let's Encrypt exigerait un nom de domaine public.

Pas d'authentification à deux facteurs , ni de quotas cumulés par utilisateur, ni de récupération de mot de passe.

Docker publie ses ports directement dans iptables : un filtrage strict des conteneurs passerait par la chaîne `DOCKER-USER`.

30.2 Les suites naturelles

— Remplacer l'ordonnanceur maison par Kubernetes, et comprendre au passage ce que Kubernetes résout vraiment.

— Ajouter de la métrologie : Prometheus pour les mesures, Grafana pour les courbes, et des alertes sur la saturation du parc.

— Centraliser les journaux des quatre nœuds, pour pouvoir enquêter sans se connecter machine par machine.

— Facturer à la minute, ce qui suppose de fiabiliser la mesure du temps réellement consommé.

30.3 Le dépôt

https://github.com/aziz3r/insacloud

Ce guide décrit l'état du projet au 13 octobre 2026.
Les extraits de code sont tirés du dépôt sans retouche.